
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Прикладной Математики и Информатики
Кафедра интеллектуальных систем

Направление подготовки / специальность: 09.04.01 Информатика и вычислительная техника

Направленность (профиль) подготовки: Прикладная математика и информатика

**ПРИМЕНЕНИЕ МЕТОДОВ КАУЗАЛЬНОГО ВЫВОДА ПРИ
АНАЛИЗЕ ЗАВИСИМОСТЕЙ МЕЖДУ
ОПТИМИЗИРУЮЩИМИ ПРОХОДАМИ LLVM-
КОМПИЛЯТОРА**

(магистерская диссертация)

Студент:

Белонин Владимир Викторович

(подпись студента)

Научный руководитель:

Дулин Сергей Константинович,
д-р техн. наук, проф.

(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2026

Применение методов каузального вывода при анализе зависимостей между оптимизирующими проходами LLVM-компилятора

Современные оптимизирующие компиляторы предоставляют десятки и сотни оптимизирующих проходов, эффект которых существенно зависит от порядка применения. Подбор оптимального порядка — известная задача *phase ordering problem* (далее в работе — проблема поиска порядка применения оптимизаций) — решается на практике эвристически (фиксированные уровни -O2, -Oz) либо методами машинного обучения, которые рассматривают компилятор как чёрный ящик и не объясняют, почему тот или иной порядок проходов оказывается выгодным. Существующие подходы анализируют корреляции между порядком проходов и итоговым качеством кода, что не позволяет отделить причинно-следственные зависимости от случайных совпадений.

В работе предлагается метод обнаружения каузальных зависимостей между оптимизирующими проходами LLVM-компилятора на основе рандомизированного эксперимента. Случайные перестановки набора из 89 проходов на выборке из 517 бенчмарков образуют рандомизированный эксперимент в смысле Рубина–Неймана: средний эффект порядка пары проходов (Average Treatment Effect) совпадает с её каузальным эффектом без необходимости устранения конфаундеров методами поиска структуры. На основе 5,17 миллиона прогонов построены каузальные графы зависимостей между проходами на трёх уровнях агрегации — глобальный, по доменным классам программ и по кластерам, выделенным по структуре промежуточного представления.

Показано, что 92,7% статистически значимых корреляционных рёбер между порядком пар проходов и метрикой улучшения кода не имеют каузальной природы; каузальный граф из 608 рёбер существенно беднее корреляционного из 5124 рёбер, построенного на тех же данных. Эффект порядка пар проходов гетерогенен по типам программ: до 29% рёбер глобального графа меняют знак на отдельных кластерах. Дополнительно исследованы две

формулировки воздействия — относительный порядок пары проходов и их непосредственное соседство, — выявляющие глобальные и локальные классы зависимостей. Собранные датасеты (517 скомпилированных бенчмарков, 5,17 миллиона прогонов с метриками) представляют самостоятельный научный вклад работы.

Содержание

1	Введение	8
1.1	Актуальность	8
1.2	Цель работы	9
1.3	Научная новизна	10
1.4	Положения, выносимые на защиту	12
2	Обзор литературы	14
2.1	Задача поиска порядка применения оптимизаций	14
2.2	Методы машинного обучения для автотюнинга компиляторов	15
2.3	Корреляционные подходы и ограничения	16
2.4	Причинно-следственный вывод	17
2.5	Каузальный вывод в смежных областях компьютерных наук	18
2.6	Позиционирование настоящей работы	18
3	Теоретические основы	20
3.1	Оптимизирующие проходы в компиляторе LLVM	20
3.1.1	Промежуточное представление и архитектура проходов	20
3.1.2	Категории проходов	21
3.2	Каузальный вывод: модель потенциальных исходов	22
3.2.1	Постановка задачи каузального вывода	22
3.2.2	Потенциальные исходы и средний эффект воздействия	23
3.2.3	Рандомизированный эксперимент	24
3.2.4	Условный средний эффект воздействия	24
3.3	РС-алгоритм поиска каузальной структуры	25
3.4	Статистические методы	26
3.4.1	Тест Уэлча для разности средних	26
3.4.2	Множественное тестирование и коррекция Бенджамини–Хохберга	27

3.4.3	Размер эффекта	27
4	Предлагаемый метод	29
4.1	Постановка задачи каузального анализа	29
4.1.1	Единица наблюдения, воздействие и исход	29
4.1.2	Обоснование выбора нормированного исхода	30
4.1.3	Применимость рандомизированного эксперимента	31
4.1.4	Две формулировки воздействия	32
4.2	Этапы каузального анализа	33
4.2.1	Шаг 1. Вычисление АТЕ для всех пар проходов	33
4.2.2	Шаг 2. Коррекция множественного тестирования	34
4.2.3	Шаг 3. Построение каузального графа	34
4.3	Уровни агрегации	35
4.3.1	Глобальный уровень	35
4.3.2	Уровень доменных классов (per-class)	35
4.3.3	Уровень структурных кластеров (per-cluster)	36
4.4	Группировки программ	36
4.4.1	Ручная классификация (per-class)	36
4.4.2	Автоматическая кластеризация (per-cluster)	37
4.5	Условный средний эффект воздействия	38
4.6	Корреляционный граф для сравнения	39
4.6.1	Определение корреляционного графа	39
4.6.2	Обоснование выбора метода	40
4.6.3	Сравнение каузального и корреляционного графов	40
5	Эксперимент	42
5.1	Инфраструктура и среда выполнения	42
5.2	Тестовый набор бенчмарков	43
5.2.1	Набор lac-dcc	43
5.2.2	Набор AnghaBench	43

5.2.3	Компиляция в промежуточное представление	43
5.3	Подсчёт числа IR-инструкций	44
5.4	Дизайн эксперимента	44
5.4.1	Параметры прогонов	44
5.4.2	Опорные значения	45
5.4.3	Выполнение и сохранение результатов	45
5.5	Статистическая мощность	46
5.5.1	Вариант 1 (относительный порядок)	46
5.5.2	Вариант 2 (соседство)	47
6	Результаты	48
6.1	Неоптимальность стандартного пайплайна $-Oz$	48
6.2	Глобальный каузальный граф	48
6.2.1	Топ-10 рёбер по величине эффекта	49
6.2.2	Инженерная интерпретация топ-10 рёбер	50
6.2.3	Структура графа: источники, стоки, паттерн «расширение– очистка»	51
6.2.4	Величина эффектов	53
6.3	Содержательный каузальный граф	53
6.3.1	Асимметрия метрики и отключающие эффекты	53
6.3.2	Топ-10 содержательных рёбер	55
6.3.3	Согласованность с пайплайном $-Oz$	56
6.4	Каузальные графы на уровне доменных классов	57
6.5	Каузальные графы на уровне структурных кластеров	58
6.6	Гетерогенность каузальных эффектов (CATE)	59
6.6.1	Рёбра с противоположным знаком на подгруппах	59
6.7	Сравнение двух формулировок воздействия	60
6.8	Сравнение каузального и корреляционного графов	62
6.8.1	Сравнение по подгруппам	63

6.8.2	Содержательная интерпретация расхождения	63
7	Заключение	65
Приложение А.	Полный список оптимизирующих проходов	74

1 Введение

1.1 Актуальность

Современные оптимизирующие компиляторы достигают высокой эффективности генерируемого кода за счёт последовательного применения десятков и сотен независимых оптимизирующих проходов. В компиляторе LLVM [1] число доступных проходов превышает 150 [2], а итоговое качество кода — размер бинарного файла, число инструкций промежуточного представления, время выполнения — существенно зависит не только от выбора подмножества применяемых проходов, но и от порядка их применения. Эта зависимость известна в литературе как *проблема поиска порядка применения оптимизаций* [3, 4, 5].

Сложность задачи определяется комбинаторным взрывом: для последовательности длины L из набора в N проходов число различных перестановок составляет $\frac{N!}{(N-L)!}$, что при $N = 50$ и $L = 33$ (длина пайплайна -Oz в LLVM 16) превосходит 10^{49} . Полный перебор пространства последовательностей невозможен даже для существенно меньших N и L [5]. На практике компиляторы используют фиксированные уровни оптимизации (-O0, -O1, -O2, -O3, -Oz), формирующие универсальную последовательность проходов, выбранную разработчиками компилятора. Эмпирически показано, что фиксированные уровни оптимизации не являются оптимальными: для конкретных программ поиск специализированных последовательностей проходов позволяет получать существенный выигрыш относительно фиксированных уровней по различным метрикам качества кода [6, 7, 8].

Существующие подходы к решению задачи поиска порядка применения оптимизаций можно разделить на три категории. *Итеративная компиляция* [3, 4] ищет оптимальные последовательности через прямой перебор, генетические алгоритмы или случайный поиск; такие методы вы-

числительно дороги и не переносят знание между программами. *Методы машинного обучения* [2, 9, 10, 7, 11, 8] обучают модель предсказывать оптимальную последовательность по признакам входной программы; такие методы обеспечивают перенос знания, однако работают как чёрный ящик, не предоставляя объяснения, почему конкретный порядок проходов оказывается выгодным. В частности, недавние работы используют большие языковые модели для генерации «causal rationales» [8], но интерпретируемость в них достигается через семантические рассуждения модели, а не через статистические оценки эффектов в рамках рандомизированных экспериментов. *Корреляционный анализ* строит зависимости между присутствием пар проходов и итоговым качеством кода, но корреляция не отделяет причинно-следственные эффекты от ложных совпадений и не позволяет ответить на вопрос: «если поставить проход А перед проходом В, изменится ли среднее качество кода?» [12].

Перечисленные подходы не описывают структуру пространства оптимизаций в терминах формальных каузальных зависимостей между отдельными парами проходов. В то же время аппарат причинно-следственного вывода, развитый в работах Рубина и Перла [13, 14, 15, 16], успешно применяется в смежных областях компьютерных наук — анализе производительности конфигурируемых программных систем [17, 18], тестировании [19] — но систематически не применялся к задаче поиска порядка применения оптимизаций. Заполнение этого пробела является актуальной научной задачей, ответ на которую открывает путь к интерпретируемым методам автотюнинга компиляторов.

1.2 Цель работы

Целью работы является выявление каузальных зависимостей между оптимизирующими проходами LLVM-компилятора и построение интерпретиру-

емых графов их взаимодействий — с отделением причинно-следственных связей от корреляционных — для понимания структуры пространства оптимизаций.

Для достижения цели поставлены следующие задачи:

1. Сформировать набор оптимизирующих проходов и набор тестовых программ, обеспечивающих репрезентативное покрытие задач компилятора.
2. Спроектировать и провести масштабный рандомизированный эксперимент: выполнить случайные перестановки оптимизирующих проходов на тестовом наборе и собрать значения метрики качества кода.
3. Разработать программный pipeline каузального анализа, реализующий вычисление средних эффектов воздействия (ATE), коррекцию множественного тестирования и построение каузальных графов.
4. Построить каузальные графы на трёх уровнях агрегации: глобальный, по доменным классам программ и по кластерам, выделенным по структуре промежуточного представления.
5. Исследовать гетерогенность эффектов с помощью условного среднего эффекта воздействия (CATE) и сравнить две формулировки воздействия: относительный порядок пары проходов и их непосредственное соседство.
6. Построить корреляционный граф на тех же данных и количественно сравнить его с каузальным графом для оценки доли ложных рёбер в корреляционном анализе.

1.3 Научная новизна

Научная новизна работы определяется следующими положениями.

1. Предложено применение формального аппарата причинно-следственного вывода в рамках модели потенциальных исходов к задаче поиска порядка применения оптимизаций в LLVM-компиляторе. Существующие работы, использующие термин «каузальный» применительно к оптимизации компиляторов [8], опираются на семантические рассуждения языковых моделей, а не на математический аппарат рандомизированных экспериментов и оценок АТЕ.
2. Предложена методика построения каузальных графов зависимостей между парами оптимизирующих проходов на основе рандомизированного эксперимента, не требующая алгоритмов поиска структуры (РС-алгоритм и его аналоги): рандомизация порядка проходов автоматически устраняет конфаундеры.
3. Получена количественная оценка доли ложных рёбер в корреляционном анализе зависимостей между проходами LLVM (92,7% корреляционных рёбер не имеют каузальной природы), что обосновывает необходимость каузального подхода.
4. Продемонстрирована гетерогенность каузальных эффектов: до 29% рёбер глобального графа меняют знак на отдельных кластерах программ, что обосновывает построение специализированных графов на уровне домена и структуры IR.
5. Собранные датасеты — 517 бенчмарков, скомпилированных в бинарное представление LLVM IR, и 5,17 миллиона прогонов с измеренными значениями метрики — являются дополнительным научным вкладом работы и могут быть использованы в последующих исследованиях.

1.4 Положения, выносимые на защиту

1. Случайные перестановки оптимизирующих проходов LLVM образуют рандомизированный эксперимент, в котором средний эффект порядка пары проходов (ATE) совпадает с её каузальным эффектом без необходимости устранения конфаундеров алгоритмами поиска структуры.
2. Построение каузальных графов зависимостей между парами проходов на нескольких уровнях агрегации (глобальном, доменных классов и структурных кластеров) даёт интерпретируемое описание структуры пространства оптимизаций: обнаруженные рёбра воспроизводят инженерные паттерны стандартного пайплайна `-Oz` (в частности, фрагмент `gvn → instcombine → dse`).
3. Корреляционный анализ зависимостей между порядком пар проходов и улучшением метрики систематически переоценивает число таких зависимостей по сравнению с каузальным анализом на одних и тех же данных, поскольку относит к зависимостям ложные совпадения. На собранном наборе данных (517 программ, 89 проходов, метрика числа инструкций промежуточного представления) каузальный граф содержит 608 рёбер против 5124 рёбер корреляционного, то есть 92,7% значимых корреляций не имеют каузальной природы; конкретное значение доли зависит от выборки программ, набора проходов, метрики и порогов значимости, тогда как направление вывода определяется самой природой корреляции и устойчиво относительно этих условий.
4. Эффект порядка пар проходов гетерогенен по типам программ: в проведённом эксперименте до 29% рёбер глобального каузального графа меняют знак на отдельных кластерах программ, что обосновывает построение специализированных каузальных графов на уровне домена и структуры промежуточного представления.

5. Две формулировки воздействия — относительный порядок («проход A где-то раньше прохода B ») и непосредственное соседство («проход A непосредственно перед проходом B ») — выявляют разные классы зависимостей: глобальные и локальные синергии соответственно; на собранных данных это даёт 608 и 44 ребра глобального графа.

2 Обзор литературы

В обзоре последовательно рассматриваются: история и масштаб задачи поиска порядка применения оптимизаций, методы машинного обучения для автотюнинга компиляторов, ограничения корреляционных подходов, теория причинно-следственного вывода и применение каузальных методов в смежных областях компьютерных наук. Завершает главу позиционирование настоящей работы относительно существующих результатов.

2.1 Задача поиска порядка применения оптимизаций

Зависимость качества генерируемого кода от порядка применения оптимизирующих проходов известна с конца 1990-х годов. В работе [3] продемонстрировано, что специализированные последовательности проходов, найденные генетическим алгоритмом, дают существенное уменьшение размера кода относительно фиксированного пайплайна. В работе [4] систематически исследовано пространство компиляционных последовательностей и показано, что эффективная последовательность зависит от программы и её свойств.

В работе [5] проведён полный перебор пространства порядков для ограниченного набора проходов и программ; даны количественные оценки масштаба пространства поиска и показано, что рассматриваемая задача не сводима к локальным эвристикам. В [20] концептуально обоснована необходимость адаптивных компиляторов, способных подбирать последовательность под входную программу.

Современное состояние задачи и его масштаб документированы в обзоре [2], где рассматриваемая задача выделена как одно из двух центральных направлений автотюнинга компиляторов наряду с задачей выбора подмножества оптимизаций (optimization selection). В обзоре указано: число оптимизирующих проходов в LLVM превышает 150, в GCC — 200;

пространство последовательностей длины несколько десятков превосходит 10^{60} перестановок и не поддаётся полному перебору. Для конкретных программ улучшение специализированной последовательности относительно -O3/-Oz достигает десятков процентов.

2.2 Методы машинного обучения для автотюнинга компиляторов

С учётом невозможности полного перебора, основным практическим подходом к задаче поиска порядка применения оптимизаций стало машинное обучение. Обзор [21] систематизирует развитие области с 2000-х годов: рассматриваются признаковые представления программ, модели (SVM, нейронные сети, обучение с подкреплением), стратегии обучения и развёртывания. Выделена общая схема «feature engineering → learning a model → deployment», в которой компилятор остаётся «чёрным ящиком»: модель предсказывает оптимальную последовательность проходов, не давая объяснения, почему именно эта последовательность выгодна.

Одной из первых самонастраивающихся систем стала Milepost GCC [9], обучающая модель предсказания флагов оптимизации по статическим признакам программы. В работе [22] предложен универсальный фреймворк OpenTuner, объединяющий несколько алгоритмов поиска. В [6] эта задача формализована как марковский процесс принятия решений: на каждом шаге политика выбирает следующий проход на основе текущего состояния кода; обучена нейронная сеть через NEAT (NeuroEvolution of Augmenting Topologies); продемонстрировано ускорение относительно -O3 на отдельных бенчмарках.

Развитие методов обучения с подкреплением для этой задачи представлено в [10, 23] (система AutoPhase): PPO-агент выбирает следующий проход на основе признаков IR. В диссертации [10] также представлен

NeuroVectorizer — модель, предсказывающая параметры векторизации в LLVM.

В последние годы крупные индустриальные системы получили развёрнутые ML-фреймворки. MLGO [11] (Google) интегрирует ML-управление в конкретные оптимизации LLVM (inlining, register allocation). CompilerGym [24] (Meta) предоставляет стандартизованное RL-окружение для исследований в области ML-компиляторов. В работе [7] CompilerGym используется для обучения политики выбора последовательностей из опорного набора — 50 предварительно отобранных «универсальных» последовательностей длины 12–13, — что даёт улучшение $-4,5\%$ по размеру кода в среднем по 4683 программам относительно -Oz.

Для эффективного признакового представления программ в [25] предложен ProGraML — графовое представление, кодирующее поток управления, поток данных и иерархию типов в едином графе и используемое в качестве входа графовых нейронных сетей.

2.3 Корреляционные подходы и ограничения

Часть существующих работ опирается на корреляционный анализ: исследуется зависимость качества кода от присутствия отдельных проходов или их пар. В диссертации [10] приведён раздел «Correlation of Passes and Program Features», в котором рассчитываются корреляции между парами проходов и метриками. Корреляционный анализ позволяет идентифицировать ассоциации, однако не отделяет причинно-следственные эффекты от ложных совпадений: если два прохода A и B оба обычно применяются в составе общей последовательности, между ними возникнет высокая корреляция, не несущая каузального содержания [12].

Ограничение корреляционного анализа особенно проявляется в задачах с большим числом переменных и сложной структурой пайплайнов: чис-

ло статистически значимых корреляций может быть сопоставимо с полным числом пар, что делает корреляционный граф непригодным для построения интерпретируемой структуры зависимостей. Количественная оценка этого эффекта для задачи поиска порядка применения оптимизаций ранее не приводилась; в настоящей работе показано, что 92,7% значимых корреляций между порядком пар проходов и улучшением метрики не имеют каузальной природы (см. **Главу 6**).

2.4 Причинно-следственный вывод

Современная теория причинно-следственного вывода развивалась в двух дополняющих друг друга направлениях. Модель потенциальных исходов была предложена в работе [26] и развита в [13]; систематическое изложение для приложений в статистике, общественных и биомедицинских науках приведено в [16]. Альтернативное современное изложение, доступное в открытом доступе, представлено в [27].

Структурный подход на основе ориентированных ациклических графов и до-исчисления развит в [14, 15, 28]; популярное изложение приведено в [12]. Современный обзор инструментов и методов каузального вывода с уклоном в интеллектуальный анализ данных приведён в [29], где систематизируются две связанные задачи — *causal discovery* (восстановление структуры) и *causal inference* (оценка эффекта при известной структуре).

Алгоритмический подход к восстановлению каузальной структуры из наблюдаемых данных представлен РС-алгоритмом [30], восстанавливающим класс эквивалентности ориентированных ациклических графов на основе тестов условной независимости. РС-алгоритм и его модификации применимы в случаях, когда рандомизированный эксперимент невозможен; в настоящей работе рандомизация порядка проходов делает применение РС-алгоритма для построения структуры избыточным.

2.5 Каузальный вывод в смежных областях компьютерных наук

Применение методов каузального вывода в анализе программных систем — активная область исследований последних лет. В работе [17] предложена система Unicorn для каузального анализа производительности конфигурируемых программных систем: построение каузального графа зависимостей между параметрами конфигурации и метриками производительности используется для отладки регрессий и поиска оптимальных конфигураций. В [18] развивается каузальный аппарат для анализа взаимодействий feature flags. В [19] каузальное тестирование применяется для поиска корневых причин дефектов.

Перечисленные работы демонстрируют состоятельность каузального подхода в анализе сложных программных систем с большим числом дискретных параметров. Задача поиска порядка применения оптимизаций представляет собой систему аналогичной структуры: дискретные переменные (порядок проходов) и количественный исход (метрика качества кода). Однако формальное применение аппарата причинно-следственного вывода к данной задаче ранее не предпринималось.

2.6 Позиционирование настоящей работы

Настоящая работа заполняет пробел между двумя направлениями исследований: с одной стороны, существуют методы машинного обучения для задачи поиска порядка применения оптимизаций, не предоставляющие интерпретируемой структуры пространства оптимизаций; с другой — развит формальный аппарат причинно-следственного вывода, успешно применяющийся в смежных областях анализа программных систем, но не применявшийся к задаче порядка оптимизирующих проходов.

В отличие от существующих ML-подходов [6, 10, 7, 11], использующих

компилятор как чёрный ящик, в настоящей работе строится интерпретируемая структура каузальных зависимостей между парами проходов в виде графа. В отличие от корреляционных анализов [10], в настоящей работе количественно отделяются каузальные эффекты от ложных корреляций. В отличие от работы ЕССО [8], использующей термин «каузальный» в неформальном смысле, в настоящей работе применяется формальный аппарат рандомизированного эксперимента и оценок АТЕ.

Настоящая работа развивает направление, заданное работами по причинно-следственному анализу конфигурируемых программных систем [17, 18, 19], переносит его методологию на эту задачу и расширяет её аппаратом гетерогенности эффектов (САТЕ) и сравнения формулировок воздействия.

3 Теоретические основы

В настоящей главе изложены теоретические концепции, на которые опирается работа: архитектура оптимизирующих проходов LLVM-компилятора, модель потенциальных исходов и средних эффектов воздействия, краткие сведения о РС-алгоритме поиска каузальной структуры, а также используемые статистические методы.

3.1 Оптимизирующие проходы в компиляторе LLVM

3.1.1 Промежуточное представление и архитектура проходов

Компилятор LLVM [1] построен вокруг типизированного промежуточного представления (*Intermediate Representation*, IR), сохраняемого в форме статического одиночного присваивания (Static Single Assignment, SSA). Программа в IR состоит из модулей, содержащих определения функций; функция разбита на базовые блоки — последовательности инструкций без переходов внутри, — образующие граф потока управления. Набор инструкций IR близок к RISC-архитектуре и насчитывает около 60 опкодов, описанных в спецификации языка [31].

Оптимизации компилятора реализованы в виде *проходов* (passes) — независимых модулей, выполняющих преобразования над IR. Различают анализирующие проходы (вычисляющие свойства программы без её модификации) и трансформирующие. По области действия проходы делятся на проходы уровня модуля, функции, цикла и базового блока. На текущий момент LLVM включает более 150 оптимизирующих проходов [2, 32].

Управление выполнением проходов осуществляет *pass manager*. В LLVM 16 сосуществуют две реализации: *legacy pass manager*, выполняющий проходы строго в заданном пользователем порядке, и *new pass manager*, использующий собственное планирование с инвалидацией результатов анализа [32].

В настоящей работе используется `legacy pass manager`, поскольку каузальный вопрос о влиянии порядка проходов теряет смысл, если `pass manager` перепланирует проходы.

Уровни оптимизации (`-O0`, `-O1`, `-O2`, `-O3`, `-Os`, `-Oz`) представляют собой предопределённые последовательности проходов, выбранные разработчиками LLVM для типичных сценариев использования. Размерно-ориентированными являются уровни `-Os` и `-Oz`. Уровень `-Oz` не сводится к чистой минимизации размера кода: по своему составу он представляет собой компромиссный режим относительно `-O2`, в котором отключены агрессивные проходы, приводящие к разрастанию размера (агрессивный инлайнинг, развёртка циклов, векторизация), тогда как прочие оптимизации сохраняются. В настоящей работе `-Oz` используется как эталон для сравнения, поскольку из стандартных уровней он в наибольшей степени ориентирован на ту же метрику, что и основная метрика работы — число IR-инструкций.

3.1.2 Категории проходов

В работе используется набор из 89 оптимизирующих проходов LLVM, объединяющий проходы пайплайна `-Oz` и расширение, отобранное по семантической значимости. Полный список проходов приведён в приложении; здесь укажем основные категории.

- **Скалярные оптимизации:** `instcombine`, `simplifycfg`, `reassociate`, `early-cse`, `sscp`, `gvn` и т.п. — локальные преобразования над инструкциями и базовыми блоками.
- **Преобразования циклов:** `licm`, `loop-unroll`, `loop-rotate`, `indvars` и др. — оптимизации, применяемые к циклам.
- **Memory- и SSA-проходы:** `mem2reg`, `sroa`, `memcpyopt`, `dse`, `adce`, `bdce` — преобразования, работающие с SSA-формой и обращениями

к памяти.

- **Межпроцедурные оптимизации:** `inline`, `always-inline`, `partial-inline`, `globalopt`, `globaldce`, `ipscpp` — проходы уровня модуля.
- **Векторизация:** `loop-vectorize`, `slp-vectorizer`, `vector-combine`, `load-store-vectorizer`.
- **Очистка и нормализация:** `strip-dead-prototypes`, `dce`, `constmerge`, `mergefunc`.

Принадлежность прохода к категории определяет ожидаемый характер взаимодействия с другими проходами: скалярные проходы часто образуют пары «expansion $\rightarrow$ cleanup», а проходы циклов зависят от предварительной нормализации SSA.

3.2 Каузальный вывод: модель потенциальных исходов

3.2.1 Постановка задачи каузального вывода

Каузальный вывод [14, 16, 27] ставит задачу количественной оценки причинно-следственного эффекта одного события на другое. В отличие от ассоциативной (корреляционной) статистики, отвечающей на вопрос «как изменяется распределение Y при наблюдаемом изменении X ?», каузальный вывод отвечает на интервенционный вопрос «как изменилось бы распределение Y , если бы X было изменено внешним вмешательством?» [12]. Различие принципиально: при наличии общего конфаундера Z , влияющего и на X , и на Y , наблюдаемая корреляция $\text{corr}(X, Y)$ может быть высокой, тогда как причинно-следственный эффект X на Y равен нулю.

В работе используется *модель потенциальных исходов* (potential outcomes framework), предложенная Нейманом [26] и развитая Рубином [13, 16]. Под-

ход формализует каузальный эффект в терминах гипотетических наблюдений, что отличает его от структурного подхода Перла [15], использующего ориентированные ациклические графы. Оба подхода математически эквивалентны [28]; в настоящей работе предпочтена модель потенциальных исходов как более естественная для рандомизированных экспериментов.

3.2.2 Потенциальные исходы и средний эффект воздействия

Пусть для каждой единицы наблюдения i определена бинарная переменная воздействия (treatment) $T_i \in \{0, 1\}$ и наблюдаемый исход (outcome) Y_i . Модель потенциальных исходов вводит для каждой единицы i два потенциальных исхода:

- $Y_i(1)$ — значение исхода, которое было бы наблюдаено, если бы единица i получила воздействие ($T_i = 1$);
- $Y_i(0)$ — значение исхода при отсутствии воздействия ($T_i = 0$).

Индивидуальный каузальный эффект определяется как разность $\tau_i = Y_i(1) - Y_i(0)$. Фундаментальная проблема каузального вывода [13] состоит в том, что для каждой единицы i наблюдается ровно одно из двух значений: $Y_i = T_i \cdot Y_i(1) + (1 - T_i) \cdot Y_i(0)$. Поэтому индивидуальный эффект ненаблюдаем; идентифицируемой характеристикой является *средний эффект воздействия* (Average Treatment Effect):

$$\text{ATE} = \mathbb{E}[Y_i(1) - Y_i(0)] = \mathbb{E}[Y_i(1)] - \mathbb{E}[Y_i(0)]. \quad (1)$$

Наивная оценка

$$\widehat{\text{ATE}}_{\text{naive}} = \mathbb{E}[Y_i \mid T_i = 1] - \mathbb{E}[Y_i \mid T_i = 0] \quad (2)$$

является несмещённой оценкой АТЕ при выполнении условия *независимости назначения воздействия от потенциальных исходов*: $\{Y_i(1), Y_i(0)\} \perp$

T_i . Для произвольных наблюдаемых данных это условие, как правило, нарушается из-за наличия конфаундеров; для его обеспечения требуется либо явное моделирование (propensity score, регрессионная корректировка), либо постановка *рандомизированного эксперимента*, в котором значение T_i присваивается случайно и независимо от характеристик единицы i .

3.2.3 Рандомизированный эксперимент

В рандомизированном эксперименте [26, 16] назначение воздействия T_i осуществляется по случайному правилу, не зависящему от единицы i и её потенциальных исходов. Пусть T_i — случайная величина, не зависящая от $\{Y_i(1), Y_i(0)\}$; тогда условие независимости выполнено по построению, и наивная оценка (2) является несмещённой оценкой АТЕ:

$$\mathbb{E}[Y_i \mid T_i = 1] = \mathbb{E}[Y_i(1) \mid T_i = 1] = \mathbb{E}[Y_i(1)], \quad (3)$$

$$\mathbb{E}[Y_i \mid T_i = 0] = \mathbb{E}[Y_i(0) \mid T_i = 0] = \mathbb{E}[Y_i(0)], \quad (4)$$

откуда $\mathbb{E}[\widehat{\text{ATE}}_{\text{naive}}] = \text{ATE}$.

Это утверждение является ключевым теоретическим обоснованием подхода настоящей работы. В **Главе 4** показано, что случайные перестановки оптимизирующих проходов LLVM удовлетворяют условию рандомизации для каждой пары проходов, и, следовательно, АТЕ на полученных данных совпадает с каузальным эффектом порядка пары проходов на метрику качества кода.

3.2.4 Условный средний эффект воздействия

Допущение, что эффект воздействия одинаков для всех единиц, в реальных данных часто нарушается: одно и то же воздействие может оказывать разный эффект на различных подгруппах единиц. Для описания такой ге-

терогенности вводится *условный средний эффект воздействия* (Conditional Average Treatment Effect):

$$\text{CATE}(x) = \mathbb{E}[Y_i(1) - Y_i(0) \mid X_i = x], \quad (5)$$

где X_i — набор ковариат, описывающих характеристики единицы i .

В настоящей работе CATE оценивается по дискретным группам — доменным классам программ (per-class) и кластерам, выделенным по структуре промежуточного представления (per-cluster). Гетерогенность каузальных эффектов между группами интерпретируется как зависимость влияния порядка прохождений от типа оптимизируемой программы.

3.3 РС-алгоритм поиска каузальной структуры

Когда рандомизация невозможна, для восстановления каузальной структуры из наблюдаемых данных используются алгоритмы поиска структуры [30, 29]. Наиболее известный из них — РС-алгоритм (Peter–Clark), предложенный Спиртесом и Глаймуром [30].

РС-алгоритм восстанавливает класс эквивалентности ориентированных ациклических графов (CPDAG) по результатам тестов условной независимости. Алгоритм состоит из двух фаз. На первой фазе строится скелет графа: для каждой пары переменных проверяется наличие подмножества переменных, при условии на которое они становятся независимы; если такое подмножество найдено, ребро между переменными удаляется. На второй фазе ориентируются рёбра: применяются правила, выводящие ориентацию из коллайдеров (структур $X \rightarrow Z \leftarrow Y$, где X и Y независимы безусловно, но зависимы при условии Z).

В настоящей работе РС-алгоритм *не применяется* для восстановления каузальной структуры, поскольку рандомизация порядка прохождений автоматически устраняет конфаундеры (см. раздел 3.2.3). РС-алгоритм мог

бы использоваться для проверки наличия медиаторов — проходов C , опосредующих эффект пары (A, B) , — однако в компиляторных пайплайнах прямые и опосредованные эффекты могут сосуществовать (один и тот же проход может влиять на исход и непосредственно, и через подготовку IR для других проходов), и удаление ребра при обнаружении медиатора не всегда оправдано. Опциональная РС-проверка на медиаторы вынесена в задачи дальнейших исследований.

3.4 Статистические методы

3.4.1 Тест Уэлча для разности средних

Для проверки статистической значимости разности средних между группами с возможно неравными дисперсиями используется тест Уэлча [33] — модификация t -теста Стьюдента [34], не предполагающая равенства дисперсий. Статистика теста имеет вид

$$t = \frac{\bar{Y}_1 - \bar{Y}_0}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_0^2}{n_0}}}, \quad (6)$$

где $\bar{Y}_1, \bar{Y}_0$ — выборочные средние, s_1^2, s_0^2 — выборочные дисперсии, n_1, n_0 — размеры групп воздействия и контроля. При справедливости нулевой гипотезы $\mathbb{E}[Y(1)] = \mathbb{E}[Y(0)]$ статистика приближённо распределена как t -распределение Стьюдента с числом степеней свободы, вычисляемым по формуле Уэлча–Саттертуэйта.

Тест Уэлча выбран как стандартный инструмент сравнения средних в условиях, когда априорные предположения о равенстве дисперсий не оправданы; в настоящей работе дисперсии исходов в группах « A раньше B » и « B раньше A » могут различаться, поскольку разные порядки проходов по-разному взаимодействуют с остальными проходами в последовательности.

3.4.2 Множественное тестирование и коррекция Бенджамини–Хохберга

При проверке большого числа гипотез одновременно вероятность хотя бы одного ложноположительного результата быстро возрастает. Для K независимых тестов на уровне значимости α ожидаемое число ложноположительных результатов составляет $K\alpha$; в настоящей работе $K = 3916$ (число пар прохождений) и $\alpha = 0,05$ дают ожидаемое число ложных открытий ≈ 196 без коррекции.

Используется коррекция Бенджамини–Хохберга [35], контролирующая *долю ложных открытий* (False Discovery Rate, FDR) — ожидаемую долю ложноположительных среди отвергнутых гипотез. Алгоритм:

1. Упорядочить p -значения $p_{(1)} \leq p_{(2)} \leq \dots \leq p_{(K)}$;
2. Найти наибольший индекс k такой, что $p_{(k)} \leq \frac{k}{K}\alpha$;
3. Отвергнуть нулевые гипотезы для всех тестов с p -значением $\leq p_{(k)}$.

Коррекция Бенджамини–Хохберга предпочтена коррекции Бонферрони (контролирующей *вероятность хотя бы одного ложного открытия*, FWER), поскольку при $K = 3916$ Бонферрони требует $p < 0,05/3916 \approx 1,3 \cdot 10^{-5}$, что отсекает большое число истинных, но малых эффектов. ВНК-коррекция, контролирующая ожидаемую долю ложных среди значимых, обеспечивает разумный баланс мощности и контроля ошибок.

3.4.3 Размер эффекта

Статистическая значимость, определяемая p -значением, не несёт информации о практической значимости эффекта: при достаточно большой выборке статистически значимыми становятся даже малые эффекты, не имеющие

содержательного смысла. Для оценки практической значимости используется *размер эффекта* — стандартизованная разность средних. Стандартная мера размера эффекта — d Коэна [36]:

$$d = \frac{\bar{Y}_1 - \bar{Y}_0}{s_{\text{pooled}}}, \quad s_{\text{pooled}} = \sqrt{\frac{(n_1 - 1)s_1^2 + (n_0 - 1)s_0^2}{n_1 + n_0 - 2}}. \quad (7)$$

Эмпирически принятые ориентиры [36]: $|d| = 0,2$ — малый эффект, $|d| = 0,5$ — средний, $|d| = 0,8$ — большой. Нормировка на стандартное отклонение необходима в задачах, где исход Y имеет произвольные единицы и масштабы. В настоящей работе исход Y нормирован на величину метрики до оптимизации (*baseline*) и представляет собой безразмерную долю устранённых инструкций, сопоставимую между всеми бенчмарками независимо от их размера; поэтому при построении каузального графа используется непосредственно $|\text{ATE}|$ как мера практической значимости, с порогом $|\text{ATE}| \geq 0,005$, отсекающим статистически значимые, но практически несущественные рёбра. Эмпирическое обоснование выбранного порога приведено в **Главе 4**.

4 Предлагаемый метод

В настоящей главе описан метод обнаружения каузальных зависимостей между парами оптимизирующих проходов LLVM на основе рандомизированного эксперимента. Метод включает: постановку задачи каузального анализа в терминах модели потенциальных исходов, обоснование применимости рандомизированного эксперимента, этапы вычисления среднего эффекта воздействия (ATE) для всех пар проходов с коррекцией множественного тестирования, построение каузального графа на трёх уровнях агрегации, оценку гетерогенности эффектов (CATE), а также сравнение полученного каузального графа с корреляционным графом, построенным на тех же данных.

4.1 Постановка задачи каузального анализа

4.1.1 Единица наблюдения, воздействие и исход

Единицей наблюдения в работе является *прогон* — результат применения некоторой случайной последовательности оптимизирующих проходов к заданному бенчмарку. Для каждого прогона i известны: программа P_i (один из 517 бенчмарков), последовательность проходов π_i (упорядоченное подмножество из 89 доступных проходов) и итоговое значение метрики — число IR-инструкций после применения π_i .

Для каждой пары проходов (A, B) из $\binom{89}{2} = 3916$ неупорядоченных пар исследуется каузальный вопрос:

Влияет ли порядок проходов A и B внутри последовательности на итоговое значение метрики качества кода?

В терминах модели потенциальных исходов (раздел 3.2) для каждой пары вводятся:

- **Бинарное воздействие** $T \in \{0, 1\}$: переменная, кодирующая порядок проходов A и B в последовательности (точная формулировка зависит от выбранного определения воздействия, см. ниже).
- **Исход** Y : нормированное улучшение метрики, $Y = (M_{\text{raw}} - M_{\pi})/M_{\text{raw}}$, где M_{π} — число IR-инструкций после применения последовательности π , а M_{raw} — baseline без оптимизаций.
- **Ковариаты**: характеристики прогона, влияющие на исход и потенциально коррелированные с воздействием (присутствие и позиции остальных 87 проходов, длина последовательности, характеристики программы).

Каузальный эффект пары (A, B) определяется как $\text{ATE}(A \rightarrow B) = \mathbb{E}[Y(1) - Y(0)]$, где $Y(1)$ и $Y(0)$ — потенциальные исходы при $T = 1$ и $T = 0$ соответственно. В силу рандомизации последовательностей (см. раздел 4.1.3) истинный ATE совпадает с наивной оценкой $\mathbb{E}[Y \mid T = 1] - \mathbb{E}[Y \mid T = 0]$ и интерпретируется как ожидаемое изменение доли устранённых инструкций при изменении порядка A и B .

4.1.2 Обоснование выбора нормированного исхода

Выбор $Y = (M_{\text{raw}} - M_{\pi})/M_{\text{raw}}$ обусловлен следующими соображениями:

- **Сопоставимость между бенчмарками.** Абсолютное число устранённых инструкций несравнимо: на крупном бенчмарке (60 000 инструкций) уменьшение на 100 несущественно, на мелком (140 инструкций) — значительно. Нормировка на baseline переводит исход в единую безразмерную шкалу для всех 517 бенчмарков.
- **Нормировка на raw, а не на -Oz.** Использование -Oz в качестве знаменателя приводит к зависимости масштаба исхода от качества стандартного пайплайна: если -Oz на конкретном бенчмарке уже близок

к оптимуму, любые изменения относительно него выглядят малыми; raw представляет собой чистый baseline без оптимизаций и позволяет единообразно интерпретировать долю улучшения.

- **Прямая интерпретация АТЕ.** Поскольку Y — доля улучшения, оценённое значение АТЕ интерпретируется напрямую. Например, $\text{ATE} = 0,039$ означает: «порядок $A \rightarrow B$ улучшает метрику в среднем на 3,9 процентных пункта от baseline относительно порядка $B \rightarrow A$ ». Это позволяет сравнивать АТЕ между парами без дополнительной нормировки на стандартное отклонение (см. раздел 3.4.3).

4.1.3 Применимость рандомизированного эксперимента

В работе случайные последовательности проходов генерируются по фиксированной схеме: для каждого прогона длина L выбирается равномерно из $\{10, 11, \dots, 40\}$, проходы выбираются без повторений из 89 доступных, и расставляются в случайном порядке (равномерное распределение по перестановкам). Параметры выборки одинаковы для всех бенчмарков; на каждый бенчмарк приходится по 10 000 прогонов.

Для произвольной пары проходов (A, B) относительный порядок A и B в случайной последовательности π не зависит от присутствия и позиций остальных проходов. Это следует из симметрии схемы генерации относительно A и B : при условии присутствия обоих проходов оба упорядочения равновероятны ($\mathbb{P}\{A \text{ раньше } B\} = 1/2$), и эта симметрия сохраняется при произвольной фиксации присутствия и позиций остальных элементов π .

Следовательно, бинарная переменная воздействия $T_{AB}^{(1)}$, определяемая через относительный порядок, удовлетворяет условию независимости от потенциальных исходов: $\{Y(1), Y(0)\} \perp T_{AB}^{(1)}$. В соответствии с разделом 3.2.3, наивная оценка $\widehat{\text{ATE}} = \bar{Y}_1 - \bar{Y}_0$ является несмещённой оценкой каузального эффекта.

Это свойство устраняет необходимость в алгоритмах поиска структуры (РС-алгоритм, FCI и др.): ковариаты сбалансированы рандомизацией по построению, и дополнительная корректировка на конфаундеры не требуется.

4.1.4 Две формулировки воздействия

Каузальный вопрос о порядке пары проходов допускает несколько естественных формулировок бинарного воздействия. В работе рассматриваются две.

Вариант 1 (относительный порядок). Для пары (A, B) воздействие определяется только взаимным расположением проходов в последовательности:

$$T_{AB}^{(1)} = \begin{cases} 1, & \text{если } A \text{ встречается раньше } B \text{ в } \pi; \\ 0, & \text{если } B \text{ встречается раньше } A; \\ \text{не определено,} & \text{если хотя бы один из проходов отсутствует в } \pi. \end{cases}$$

Прогоны, в которых отсутствует хотя бы один из проходов пары, исключаются из выборки для данной пары. При параметрах эксперимента ($L \in [10, 40]$ из 89 проходов) доля прогонов с одновременным присутствием обоих проходов составляет около 8 %, что даёт около 400 000 наблюдений на пару из 5,17 миллиона прогонов.

Вариант 1 улавливает *глобальные* зависимости: эффект сохраняется, даже если между A и B стоит несколько других проходов. Содержательная интерпретация — «подготовка IR проходом A влияет на эффективность последующего применения B ».

Вариант 2 (непосредственное соседство). Воздействие фиксирует не только относительный порядок, но и непосредственное соседство:

$$T_{AB}^{(2)} = \begin{cases} 1, & \text{если в } \pi \text{ присутствует подпоследовательность } [A, B]; \\ 0, & \text{в противном случае.} \end{cases}$$

Вариант 2 улавливает *локальные синергии*: эффекты, возникающие только при непосредственном соседстве проходов. Доля прогонов с соседством $[A, B]$ существенно ниже (около 31 наблюдение на пару на бенчмарк, $\approx 16\,000$ наблюдений на пару суммарно), что снижает статистическую мощность по сравнению с вариантом 1, но даёт принципиально иной класс выявляемых зависимостей.

Зачем обе формулировки. Каузальный вопрос «влияет ли порядок» не имеет единственного канонического воздействия: оба варианта описывают разные классы зависимостей, и совместное их использование позволяет различить глобальные и локальные эффекты. Конкретное сравнение результатов двух формулировок приведено в **Главе 6**.

4.2 Этапы каузального анализа

Описанная ниже последовательность этапов применяется независимо к каждой из двух формулировок воздействия (1 и 2) и состоит из трёх обязательных шагов; четвёртый шаг (РС-проверка на медиаторы) рассмотрен как опциональное расширение и в настоящей работе не выполняется.

4.2.1 Шаг 1. Вычисление АТЕ для всех пар проходов

Для каждой из 3916 пар (A, B) :

1. Из общего множества прогонов отбираются прогоны, в которых присутствуют оба прохода пары (см. формулировку воздействия). Пары,

для которых суммарный размер выборки воздействия и контроля составляет менее 20 наблюдений, исключаются из анализа как недостаточные для статистического вывода.

2. Вычисляется оценка АТЕ: $\widehat{\text{ATE}}(A \rightarrow B) = \bar{Y}_1 - \bar{Y}_0$.
3. Проверяется статистическая значимость разности средних с помощью теста Уэлча (раздел 3.4.1): вычисляется t -статистика и p -значение.

Результат шага — таблица из 3916 строк с полями: пара (A, B) , оценка АТЕ, t -статистика, p -значение, размер выборки воздействия n_1 , размер выборки контроля n_0 .

4.2.2 Шаг 2. Коррекция множественного тестирования

К полученному массиву из 3916 p -значений применяется процедура Бенджамини–Хохберга (раздел 3.4.2) с уровнем FDR $\alpha = 0,05$. Результат — скорректированные p -значения p_{adj} и индикатор значимости.

Бонферрони-коррекция не используется: при $K = 3916$ и $\alpha = 0,05$ она требует $p < 1,3 \cdot 10^{-5}$, что отсекает большинство реальных эффектов малой и средней величины.

4.2.3 Шаг 3. Построение каузального графа

По таблице АТЕ строится направленный граф зависимостей $G = (V, E)$, где V — множество из 89 проходов, а ребро $(A \rightarrow B) \in E$ включается, если для пары (A, B) выполняются *оба* условия:

1. **Статистическая значимость:** $p_{\text{adj}} < 0,05$.
2. **Практическая значимость:** $|\widehat{\text{ATE}}(A \rightarrow B)| \geq \delta$, где $\delta = 0,005$.

Направление ребра определяется знаком АТЕ: при $\widehat{\text{ATE}}(A \rightarrow B) > 0$ ребро направлено от A к B (порядок $A \rightarrow B$ лучше), при отрицательном знаке — наоборот.

Обоснование порога $\delta = 0,005$. Без порога δ большое количество пар с микроскопическими, но статистически значимыми эффектами включается в граф. Эмпирический анализ показывает, что в построенной таблице АТЕ медианная величина $|d|$ Коэна для значимых пар составляет около 0,017 — в 10 раз меньше порога «малого эффекта» Коэна. Порог $\delta = 0,005$ (улучшение на 0,5 процентных пункта от baseline) был выбран после визуального анализа гистограммы $|\widehat{\text{ATE}}|$ значимых пар и соответствует «минимально интересной» практической величине эффекта для рассматриваемой метрики.

4.3 Уровни агрегации

Каузальный граф строится на трёх уровнях агрегации, отвечающих на разные исследовательские вопросы.

4.3.1 Глобальный уровень

АТЕ вычисляется по всем 517 бенчмаркам совокупно. Каждое наблюдение в выборке — один прогон одного бенчмарка; оценка АТЕ отражает усреднённый каузальный эффект порядка пары проходов по всему ансамблю программ. Глобальный граф используется как опорное представление структуры зависимостей.

4.3.2 Уровень доменных классов (per-class)

Бенчмарки разбиваются на 8 доменных классов на основе ручной классификации по вычислительному характеру (раздел «Группировки программ»

ниже): `crypto`, `numeric`, `multimedia`, `loop_bench`, `pointer_data`, `string_parse`, `systems`, `general`. Для каждого класса АТЕ вычисляется только по принадлежащим ему бенчмаркам, и строится отдельный каузальный граф. Per-class графы отражают предметную специфику: например, эффект векторизации в классе `loop_bench` (тесты циклов) ожидается более выраженным, чем в `systems` (системные приложения с преобладанием управления).

4.3.3 Уровень структурных кластеров (per-cluster)

Бенчмарки разбиваются на 6 кластеров по структуре промежуточного представления (см. раздел «Группировки программ»). Per-cluster графы отражают *объективную* группировку по характеристикам IR — в отличие от ручной классификации, не зависящую от знаний экспертизы. Сравнение per-class и per-cluster графов позволяет проверить, насколько домен программы определяет её каузальную структуру оптимизаций.

Суммарно для каждой формулировки воздействия строится $1 + 8 + 6 = 15$ каузальных графов; для двух формулировок — 30 графов.

4.4 Группировки программ

4.4.1 Ручная классификация (per-class)

Доменные классы введены на основе характера выполняемой программой задачи. Для бенчмарков из набора `lsc-dcc` (231 программа) использована оригинальная классификация наборов: `cBench security_*` отнесены к `crypto`, `PolyBench` — к `numeric`, `TSVC` — к `loop_bench` и т. д. Для функций из набора `AnghaBench` (286 функций) классификация выполнена индивидуально по имени файла и функции (имя содержит `extr_<file>_c_<function>`), а не по проекту-источнику; такой подход даёт более точное отражение характера конкретной функции, в отличие от классификации по доменному характеру всего проекта. Финальное распределение приведено в **Главе 5**.

4.4.2 Автоматическая кластеризация (per-cluster)

Структурная кластеризация выполняется в три шага.

Извлечение признаков IR. Для каждого из 517 бенчмарков из `.bc` файла без применения оптимизаций (`baseline_raw`) вычисляются 92 признака. Из них для кластеризации отбираются 21 *нормированный* признак, не зависящий от размера программы:

- Доли категорий инструкций (12 признаков): арифметические, логические, сравнения, обращения к памяти, переходы, вызовы и др.
- Доли ключевых опкодов (5): `load`, `store`, `alloca`, `getelementptr`, `br`.
- Структурные признаки (3): средний размер базового блока, средний размер функции, среднее число базовых блоков на функцию.
- Соотношения (1): `load_store_ratio`, `int_float_ratio`.

Абсолютные размеры (общее число инструкций, число функций) исключены, поскольку кластеризация по ним давала бы группировку «большие»/«маленькие» программы вместо группировки по структурным свойствам.

Снижение размерности и кластеризация. Признаки стандартизованы (`StandardScaler`), затем применён метод главных компонент: оставлено 13 компонент, объясняющих 95,4 % дисперсии. На пространстве компонент выполнена кластеризация методом k -средних.

Выбор числа кластеров. Силуэтный коэффициент монотонно возрастает с увеличением k от 0,1455 при $k = 3$ до 0,1899 при $k = 15$, что свидетельствует об отсутствии чётко выраженной кластерной структуры (данные образуют непрерывное облако). Дополнительная проверка алгоритмом

HDBSCAN подтверждает непрерывность: при минимальном размере кластера 25 выделяются лишь 3 кластера, и 75 % программ помечаются как шум.

В этих условиях выбор k определяется содержательными соображениями. Зафиксировано $k = 6$ как баланс между разрешением группировки и минимальным размером кластера: при $k = 6$ минимальный кластер содержит 22 программы, что соответствует приблизительно 220 000 наблюдений на пару (достаточно для статистической мощности per-cluster анализа). При $k = 5$ один из кластеров содержал бы лишь 7 программ, при $k = 7$ — лишь 4.

4.5 Условный средний эффект воздействия

Для оценки гетерогенности каузальных эффектов по типам программ вычисляется условный средний эффект воздействия (раздел 3.2.4):

$$\text{CATE}(A \rightarrow B \mid g) = \mathbb{E}[Y \mid T_{AB} = 1, G = g] - \mathbb{E}[Y \mid T_{AB} = 0, G = g],$$

где G — принадлежность бенчмарка к группе (доменному классу или структурному кластеру).

CATE вычисляется только для рёбер *глобального* каузального графа: для каждого из 608 значимых глобальных рёбер (см. **Главу 6**) оценивается CATE по 8 доменным классам, 6 структурным кластерам и 517 отдельным бенчмаркам. На уровне отдельного бенчмарка для оценки CATE требуется не менее 10 наблюдений в подгруппе; подгруппы меньшего размера исключаются.

Результаты CATE используются для двух целей:

- **Идентификация гетерогенности направления.** Если знак CATE на какой-либо подгруппе противоположен знаку глобального АТЕ, значит, ребро «меняет направление» на этой подгруппе. Доля таких рёбер

количественно характеризует ограниченность глобального графа.

- **Выделение универсальных и специализированных рёбер.** Рёбра с однородным САТЕ по всем группам можно считать универсальными правилами; рёбра с существенной дисперсией САТЕ между группами требуют специализированных графов на уровне группы.

4.6 Корреляционный граф для сравнения

Для количественного сравнения каузального и корреляционного подходов на одних и тех же данных строится корреляционный граф. Корреляционный анализ применяется только к формулировке воздействия 2 (соседство), которая соответствует постановке корреляционного анализа через биграммы — пары соседних проходов в последовательности.

4.6.1 Определение корреляционного графа

Для построения корреляционного графа исход бинаризуется: *выигрышным* называется прогон, в котором применённая последовательность проходов даёт меньшее число IR-инструкций, чем стандартный пайплайн -Oz:

$$W = \mathbb{I}[M_{\pi} < M_{\text{-Oz}}],$$

где $M_{\text{-Oz}}$ — число IR-инструкций после применения -Oz к тому же бенчмарку.

Для каждой упорядоченной пары проходов (A, B) (всего $89 \cdot 88 = 7832$ пары) проверяется гипотеза о статистической независимости двух бинарных переменных: $T_{AB}^{(2)}$ (наличие подпоследовательности $[A, B]$ в π) и W (выигрышность прогона). Строится таблица сопряжённости 2×2 :

	$W = 1$	$W = 0$
$T_{AB}^{(2)} = 1$	a	b
$T_{AB}^{(2)} = 0$	c	d

и применяется критерий Пирсона χ^2 для проверки независимости:

$$\chi^2 = \sum_{ij} \frac{(O_{ij} - E_{ij})^2}{E_{ij}},$$

где O_{ij} — наблюдаемые частоты в ячейке таблицы, E_{ij} — ожидаемые частоты при выполнении гипотезы независимости.

Биграммы, встречающиеся менее чем в 20 прогонах, исключаются из анализа как недостаточные для построения таблицы сопряжённости. p -значения, полученные критерием χ^2 для оставшихся пар, корректируются процедурой Бенджамини–Хохберга на уровне FDR 0,05. Ребро ($A \rightarrow B$) включается в корреляционный граф, если зависимость $T_{AB}^{(2)}$ и W статистически значима и доля выигрышных прогонов с биграммой $[A, B]$ превышает долю выигрышных среди прогонов без неё.

4.6.2 Обоснование выбора метода

Описанное определение корреляционного графа использует распространённую в работах по автотюнингу компиляторов схему: «выигрышные» последовательности (превосходящие фиксированный эталон $-Oz$) сопоставляются по частоте встречаемости отдельных переходов между проходами. Бинаризация исхода и анализ биграмм применяются в задачах автотюнинга начиная с работы Milepost GCC [9]. Сравнение с подобным эталонным корреляционным анализом позволяет продемонстрировать выигрыш формального каузального вывода над подходом, фактически применяющимся на практике для построения корреляционных графов между проходами.

4.6.3 Сравнение каузального и корреляционного графов

Корреляционный граф строится на тех же уровнях агрегации, что и каузальный: глобальный, per-class и per-cluster. Сравнение проводится по следующим показателям:

- **Число рёбер.** Превышение числа рёбер в корреляционном графе над числом рёбер в каузальном графе количественно характеризует масштаб ложных корреляций, индуцированных косвенными зависимостями.
- **Доля ложных корреляционных рёбер.** Доля рёбер корреляционного графа, отсутствующих в каузальном графе, интерпретируется как нижняя оценка доли корреляций без каузального содержания.

Полученные количественные оценки расхождения представлены в **Главе 6** и используются в работе как центральный аргумент в пользу необходимости каузального анализа.

5 Эксперимент

В настоящей главе описаны инфраструктурные и проектные решения, обеспечивающие применимость метода, изложенного в **Главе 4**. Последовательно рассмотрены: программная инфраструктура и версия компилятора, состав тестового набора бенчмарков и схема его компиляции, методика подсчёта метрики качества кода, параметры дизайна эксперимента, оценка статистической мощности используемых тестов и санитарные проверки полученных данных.

5.1 Инфраструктура и среда выполнения

В качестве среды компиляции и оптимизации используется LLVM версии 16. Принципиально важно использование *legacy pass manager*, который выполняет проходы строго в заданном пользователем порядке, в отличие от *new pass manager*, осуществляющего собственное планирование с инвалидацией результатов анализа. Применение нового pass manager сделало бы каузальный вопрос о влиянии порядка проходов некорректным, поскольку фактический порядок выполнения мог бы отличаться от заданного.

Legacy pass manager включается флагом `-enable-new-pm=0`, который добавляется первым во все вызовы `opt` с произвольной последовательностью проходов. Стандартный пайплайн `-Oz`, используемый для вычисления опорного значения `baseline_oz`, не работает в legacy pass manager в LLVM 16; для его расчёта используется new pass manager.

Программная часть эксперимента реализована на Python с использованием стандартных средств параллелизации (модуль `multiprocessing`). Каждый бенчмарк обрабатывается в отдельном процессе; результаты прогонов одного бенчмарка сохраняются в чекпоинт-файл, после чего объединяются с результатами других бенчмарков в единый набор данных формата Parquet.

5.2 Тестовый набор бенчмарков

Тестовый набор объединяет программы из двух источников и содержит 517 бенчмарков.

5.2.1 Набор `lac-dcc`

Набор `lac-dcc` [37] объединяет 28 классических наборов бенчмарков из исследовательской литературы по оптимизации кода: `cBench`, `MiBench`, `PolyBench`, `Prolangs-C`, `TSVC`, `mediabench`, `McGill`, `shootout`, `stanfordsorts`, `flops`, `olden`, `ptrdist` и др. Каждый бенчмарк представляет собой целую программу, скомпилированную из исходного кода на языке C. Всего в наборе используется 231 программа.

5.2.2 Набор `AnghaBench`

Набор `AnghaBench` [38] содержит миллионы извлечённых из открытых исходных кодов функций на языке C; каждая функция представлена как самодостаточный синтаксически корректный фрагмент. Из общего объёма набора в работе используется выборка из 286 функций, отобранных из 146 открытых проектов на GitHub. Имя каждой функции имеет вид `extr_<file>_c_<function>`, что упрощает классификацию по характеру выполняемой операции (см. раздел 4.4).

5.2.3 Компиляция в промежуточное представление

Каждый бенчмарк компилируется в файл LLVM bitcode (`.bc`) без применения оптимизирующих проходов командой:

```
clang -O0 -emit-llvm -c <source>.c -o <bench>.bc
```

Скомпилированные `.bc` файлы сохраняются в директории `benchmarks/compiled/` и используются как единая точка входа для всех последующих прогонов. Скрипты компиляции (`src/compile_benchmarks.py` и `src/`

`compile_anghabench.py`) обеспечивают идемпотентность: повторный запуск пропускает уже скомпилированные программы.

5.3 Подсчёт числа IR-инструкций

Метрика качества кода — число LLVM IR-инструкций после применения последовательности оптимизирующих проходов. Вычисляется в два шага:

1. Текстовое представление `.ll` формируется из `.bc` файла утилитой `llvm-dis`.
2. Полученный файл `.ll` парсится: подсчитываются строки внутри тел функций (между `define ... { ... }`), исключая продолжения многострочных конструкций (`switch` с множественными исходами).

В подсчёт включаются только LLVM IR-инструкции согласно категории *instructions* в спецификации языка [31]: `add`, `load`, `store`, `call`, `br`, `ret`, `alloca`, `icmp`, `getelementptr`, `phi` и др. Из подсчёта исключены: определения глобальных переменных и типов, объявления функций (`declare`), метки базовых блоков, метаданные, блоки `attributes`, закрывающие скобки.

Корректность подсчёта проверена на всех 517 бенчмарках путём сопоставления с независимой реализацией, основанной на полном перечне опкодов из спецификации языка [31].

5.4 Дизайн эксперимента

5.4.1 Параметры прогонов

Для каждого из 517 бенчмарков сгенерировано по 10 000 случайных прогонов; общее число прогонов составляет $517 \cdot 10\,000 = 5\,170\,000$. Параметры одной случайной последовательности π :

- длина L выбирается равномерно из множества $\{10, 11, \dots, 40\}$;
- проходы выбираются *без повторений* из 89 доступных проходов (см. приложение);
- выбранные проходы расставляются в случайном порядке (равномерное распределение по перестановкам).

Значение `seed` генератора случайных чисел зафиксировано для каждого бенчмарка отдельно, что обеспечивает воспроизводимость результатов и независимость порядка генерации от количества параллельных процессов.

5.4.2 Опорные значения

Для каждого бенчмарка перед запуском основного эксперимента вычисляются два опорных значения метрики:

- M_{raw} — число IR-инструкций в исходном `.bc` файле без применения каких-либо оптимизаций. Используется в знаменателе формулы исхода $Y = (M_{\text{raw}} - M_{\pi}) / M_{\text{raw}}$ (раздел 4.1.1).
- M_{Oz} — число IR-инструкций после применения стандартного пайплайна `-Oz`. Используется при построении корреляционного графа для определения выигрышности прогона (раздел 4.6).

5.4.3 Выполнение и сохранение результатов

Каждый прогон состоит из двух шагов: запуск `opt` с заданной последовательностью проходов и последующий подсчёт числа IR-инструкций в результирующем `.bc` файле. Прогон, в котором `opt` завершился с ошибкой или метрика не была успешно вычислена, помечается соответствующим статусом и исключается из дальнейшего анализа.

Результаты сохраняются в формате Apache Parquet с одной строкой на прогон и полями: имя бенчмарка, последовательность проходов (сериализованная в JSON), длина последовательности, число IR-инструкций после прогона, опорные значения M_{raw} и M_{Oz} , статус. Формат Parquet выбран как стандарт для табличных данных большого объёма с эффективным сжатием и поддержкой потоковой обработки.

Из общего числа 5 170 000 прогонов 5 169 736 (99,995 %) завершились успешно; 264 неуспешных прогона, сконцентрированных в 32 бенчмарках (преимущественно `lame`, `bc`, `miniGMG`), исключены из последующего анализа. Полученные распределения метрик содержательно корректны: число IR-инструкций M_{π} варьируется в диапазоне от 1 до 66 908 при медиане 140, опорные значения M_{raw} и M_{Oz} — в диапазонах $[3, 54\,014]$ и $[1, 27\,761]$ соответственно. Полный цикл эксперимента выполнен за 19,3 часа на 6 параллельных рабочих процессах; среднее время одного прогона составляет около 13 мс.

5.5 Статистическая мощность

Оценка минимально обнаруживаемого размера эффекта определяет, эффекты какой величины метод способен выявить при заданном уровне значимости. Для теста Уэлча на двух выборках одинакового размера n минимально обнаруживаемый размер эффекта d Коэна на уровне $\alpha = 0,05$ и мощности $1 - \beta = 0,80$ приближённо равен $d_{\min} \approx 2,8/\sqrt{n}$.

5.5.1 Вариант 1 (относительный порядок)

Доля прогонов, в которых оба прохода пары присутствуют одновременно, составляет около 8 % при параметрах эксперимента $L \in [10, 40]$ из 89 проходов. На каждый из 517 бенчмарков приходится приблизительно $10\,000 \cdot 0,08 \approx 800$ таких прогонов, или ≈ 400 наблюдений на пару в каж-

дой из двух групп воздействия.

При агрегации по 517 бенчмаркам (глобальный уровень) число наблюдений на пару составляет $\approx 207\,000$, что даёт $d_{\min} \approx 0,006$. При агрегации по доменному классу (в среднем 65 программ) — $\approx 26\,000$ наблюдений, $d_{\min} \approx 0,017$. Таким образом, на per-class и per-cluster уровнях вариант 1 надёжно ловит даже малые эффекты по шкале Коэна.

5.5.2 Вариант 2 (соседство)

Доля прогонов с конкретной биграммой существенно ниже: для пары (A, B) и параметров $L \in [10, 40]$ в среднем встречается около 31 биграммы $[A, B]$ на каждые 10 000 прогонов одного бенчмарка. Это даёт $\approx 16\,000$ наблюдений с биграммой при агрегации по всем 517 бенчмаркам и пропорционально меньшие выборки на меньших уровнях агрегации.

Уменьшенная мощность варианта 2 принципиальна: метод выявляет только сильные локальные синергии, в которых эффект непосредственного соседства проходов превышает шум. Конкретное число рёбер, проходящих коррекцию, представлено в **Главе 6**.

6 Результаты

В настоящей главе представлены результаты применения метода, изложенного в **Главе 4**, к данным эксперимента, описанного в **Главе 5**. Последовательно рассмотрены: эмпирическое подтверждение неоптимальности стандартного пайплайна $-Oz$ как мотивация дальнейшего анализа, структура глобального каузального графа, каузальные графы на уровне доменных классов и структурных кластеров, гетерогенность каузальных эффектов (CATE), сравнение двух формулировок воздействия и количественное сопоставление каузального и корреляционного графов.

6.1 Неоптимальность стандартного пайплайна $-Oz$

Прежде чем перейти к анализу каузальных зависимостей, отметим эмпирическое наблюдение, мотивирующее задачу настоящей работы. В 4,0 % из 5,17 миллиона прогонов случайная последовательность проходов даёт меньшее число IR-инструкций, чем стандартный пайплайн $-Oz$. Эти прогоны распределены по 374 бенчмаркам из 517 (72 %): на большинстве программ существуют последовательности проходов, превосходящие $-Oz$ по выбранной метрике. Полученное наблюдение количественно подтверждает известное в литературе утверждение о неоптимальности фиксированных уровней оптимизации (см. раздел 2.1) и обосновывает практическую значимость задачи поиска специализированных последовательностей.

6.2 Глобальный каузальный граф

Для формулировки воздействия 1 (относительный порядок) на глобальном уровне агрегации (по всем 517 бенчмаркам) из 3916 исследуемых пар проходов 1761 пара получает статистически значимое $p_{adj} < 0,05$ после коррекции Бенджамини–Хохберга. С дополнительным фильтром по практической

значимости $|\widehat{ATE}| \geq 0,005$ в каузальный граф включается **608 рёбер**.

6.2.1 Топ-10 рёбер по величине эффекта

В таблице 1 представлены десять рёбер с наибольшим значением $|\widehat{ATE}|$ в глобальном графе.

Таблица 1: Топ-10 рёбер полного глобального каузального графа по $|\widehat{ATE}|$, вариант 1.

#	Рёбро	$\widehat{ATE}$
1	loop-vectorize $\rightarrow$ licm	+0,039
2	loop-vectorize $\rightarrow$ mem2reg	+0,034
3	gvn $\rightarrow$ instcombine	+0,034
4	loop-vectorize $\rightarrow$ sroa	+0,034
5	loop-vectorize $\rightarrow$ gvn	+0,033
6	loop-vectorize $\rightarrow$ instcombine	+0,033
7	break-crit-edges $\rightarrow$ simplifycfg	+0,032
8	break-crit-edges $\rightarrow$ jump-threading	+0,030
9	gvn $\rightarrow$ dse	+0,026
10	loop-vectorize $\rightarrow$ simplifycfg	+0,026

В топ-10 доминируют рёбра, исходящие из проходов **loop-vectorize** (6 рёбер) и **break-crit-edges** (2 ребра) — проходов, увеличивающих число IR-инструкций при срабатывании. Их раннее расположение в последовательности систематически связано с *меньшим* итоговым числом IR-инструкций. Этот, на первый взгляд, контринтуитивный результат имеет инженерное объяснение: раннее размещение код-раздувающего прохода в случайной последовательности повышает вероятность того, что в момент его выполнения IR ещё не подготовлен необходимыми преобразованиями (например, переводом в SSA-форму), вследствие чего проход не выполняет

работы и не вызывает раздувания кода. Подробное разъяснение асимметрии данного типа приведено в разделе 6.3.

6.2.2 Инженерная интерпретация топ-10 рёбер

Многие рёбра в верхней части глобального графа имеют ясную инженерную интерпретацию: они представляют собой паттерн *расширение–очистка*, в котором проход, увеличивающий число IR-инструкций при срабатывании (`loop-vectorize`, `break-crit-edges`, `loop-unroll`), предшествует очищающему проходу (`instcombine`, `simplifycfg`, `dse`, `jump-threading`). Рассмотрим примеры из таблицы 1:

- `loop-vectorize` → `instcombine`: векторизация создаёт операции над векторными типами с локальными возможностями упрощения, которые `instcombine` затем сворачивает.
- `loop-vectorize` → `simplifycfg`: векторизация порождает новые базовые блоки (заголовок и остаточный цикл), которые `simplifycfg` объединяет и упрощает.
- `break-crit-edges` → `jump-threading`: расщепление критических рёбер CFG открывает новые возможности для `jump-threading` по упрощению цепочек условных переходов.

Описанные паттерны воспроизводят структуру стандартного пайплайна -Oz LLVM 16, в котором проходы `loop-vectorize` и `break-crit-edges` располагаются непосредственно перед сериями очищающих проходов. С точки зрения статистики и формального каузального вывода полученные рёбра корректны: они отражают воспроизводимые каузальные эффекты, измеренные в рамках рандомизированного эксперимента.

Однако направление этих рёбер также согласуется с *асимметрией* выбранной метрики — числа IR-инструкций — и порядка проходов в после-

довательности. Эта асимметрия, разобранный в разделе 6.3, не позволяет полностью отделить каузальный вклад от смещения метрики на основании одной только величины $\widehat{ATE}$. Для углублённого анализа без этой неопределённости в разделе 6.3 вводится *содержательный каузальный граф*, в котором рёбра с двойной асимметрией исключены.

6.2.3 Структура графа: источники, стоки, паттерн «расширение–очистка»

Распределение числа входящих и исходящих рёбер у узлов глобального графа резко асимметрично. Часть узлов выступает в роли *стоков* — проходов, для которых полезно стоять *после* большинства остальных проходов:

- `instcombine`: 86 входящих рёбер, 0 исходящих;
- `simplifycfg`: 83 входящих, 0 исходящих;
- `dse`: 83 входящих, 0 исходящих.

Другая часть выступает в роли *источников* — проходов, для которых характерно располагаться *раньше* большинства остальных. Основные источники — код-раздувающие проходы `loop-vectorize` (88 исходящих рёбер), `loop-unroll` (84) и `break-crit-edges` (84). Однако содержательная природа этих рёбер двойственна: как обсуждалось в подразделе «Содержательная природа рёбер», ранняя позиция код-раздувающего прохода в значительной части случаев означает *нарушение его предусловий* (отсутствие SSA-формы, неканонизированных циклов и т. п.), что приводит к фактическому «отключению» прохода. Положительный знак АТЕ для таких рёбер отражает уменьшение размера кода за счёт неработающей оптимизации, а не за счёт удачного упорядочивания.

Тем не менее общий статистический паттерн графа интерпретируется как «*расширение–очистка*» (`expansion–cleanup`): проходы-стоки (`instcombine`,

`simplifycfg, dse`) играют роль универсальных очистителей, эффективных после практически любого предшествующего преобразования. Этот паттерн воспроизводит структуру стандартных пайплайнов LLVM, где очищающие проходы запускаются многократно в разных позициях. Для извлечения содержательных перестановок без артефактов «отключения» в разделе 6.3 применяется фильтрация по источникам — рассмотрение каузального графа без рёбер от код-раздувающих проходов.

Структура полного графа визуализирована на рисунке 1: показаны 50 рёбер с наибольшим $|\widehat{\text{АТЕ}}|$. Хорошо видна доминирующая роль `loop-vectorize` как источника многочисленных «отключающих» рёбер.

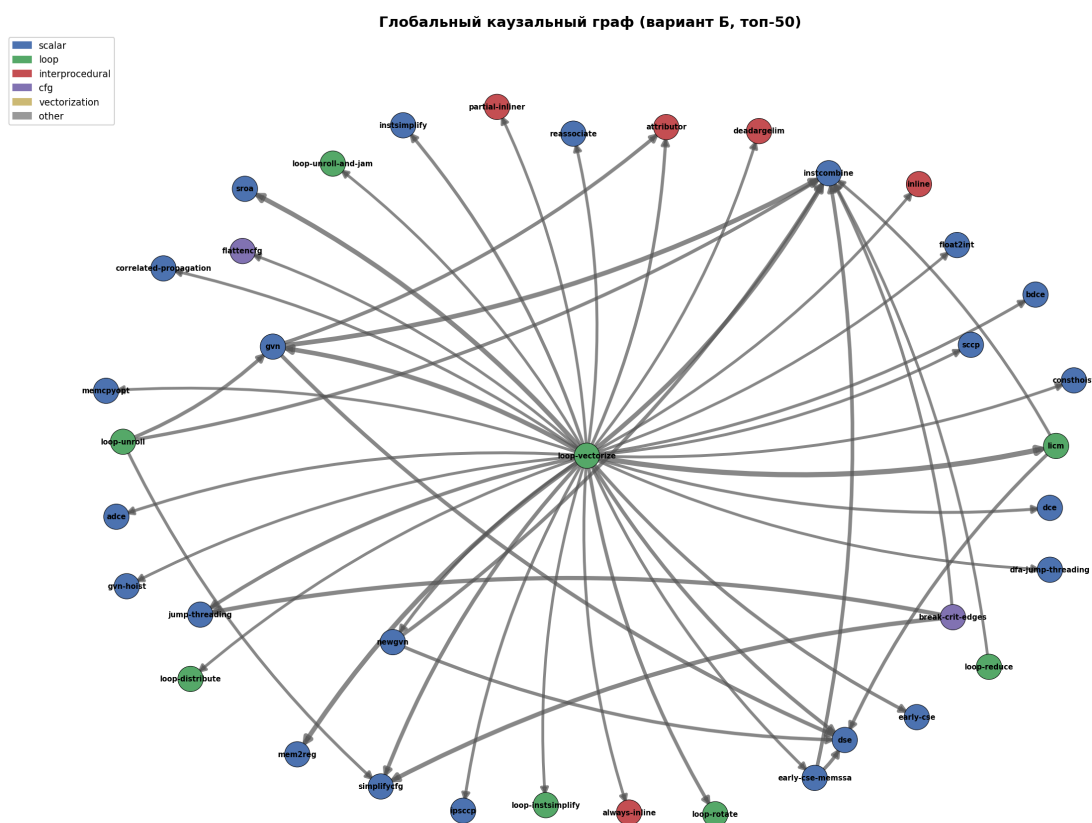


Рис. 1: Топ-50 рёбер полного глобального каузального графа (608 рёбер).

6.2.4 Величина эффектов

Медианная величина $|d|$ Коэна (см. раздел 3.4.3) для значимых рёбер глобального графа составляет около 0,017. Это на порядок меньше порога «малого эффекта» Коэна ($|d| = 0,2$); каждое отдельное ребро — статистически реальный, но *малый по абсолютной величине* эффект. Содержательная ценность каузального графа состоит не в крупных эффектах отдельных пар, а в *структуре* зависимостей и количестве согласованных направлений между проходами.

6.3 Содержательный каузальный граф

6.3.1 Асимметрия метрики и отключающие эффекты

Полный глобальный граф из 608 рёбер содержит две систематические смещения в направлениях, вызванные особенностями выбранной метрики (числа IR-инструкций) и параметрами эксперимента:

- *Смещение по источнику.* Для прохода, увеличивающего число инструкций при срабатывании (*раздуватель*: `loop-vectorize`, `loop-unroll`, `inline` и др.), раннее расположение в последовательности систематически даёт меньшее количество инструкций: ранний раздуватель работает на меньшем количестве предварительно подготовленного кода, либо не находит работы вовсе и не вызывает раздувания.
- *Смещение по цели.* Для прохода, уменьшающего число инструкций при срабатывании (*сократитель*: `instcombine`, `dse`, `simplifcfg`, `gvn`, `jump-threading` и др.), позднее расположение в последовательности систематически даёт меньшее количество инструкций: поздний сократитель работает на накопленном после всех предшествующих преобразований коде.

Рёбра вида $E \rightarrow R$ (раздуватель $\rightarrow$ сократитель) демонстрируют *двойное смещение*: оба прохода в своих позициях работают в направлении уменьшения метрики независимо от наличия реального каузального эффекта между ними. Это не означает, что такие рёбра являются артефактом: в разделе 6.2.2 показано, что значительная часть этих рёбер соответствует инженерному паттерну «расширение–очистка» из стандартных пайплайнов LLVM. Однако смещение метрики делает невозможным количественное отделение каузального вклада от смещения только по величине $\widehat{ATE}$.

Фильтрация для содержательного графа. Чтобы провести анализ без неопределённости, связанной со смещением метрики, из полного графа исключаются все рёбра $E \rightarrow R$. Это исключение охватывает 187 рёбер.

Дополнительно из дальнейшего анализа исключаются все рёбра, в которых участвует проход `loop-vectorize`: в полученном после первой фильтрации графе он остаётся универсальным источником с 78 исходящими рёбрами и нулевым числом входящих, что отражает доминирующую роль смещения метрики над содержательным каузальным эффектом. Среди всех проходов в наборе `loop-vectorize` обладает наиболее жёсткой комбинацией предусловий (SSA-форма, скаляризация агрегатов, канонизация и нумерация значений в циклах) и наибольшим коэффициентом раздувания кода при срабатывании; в случайной последовательности из 10–40 проходов вероятность одновременного выполнения всех его предусловий в правильном порядке невелика, и его эффект, регистрируемый в каузальной оценке, в значительной мере отражает факт несрабатывания, а не содержательное взаимодействие с другими проходами. Поскольку в данном случае разделение содержательного и метрического вкладов затруднено, рёбра с участием `loop-vectorize` исключаются для дальнейшего рассмотрения; это позволяет сосредоточить анализ на других проходах с более интерпретируемой

структурой эффектов.

После исключения 187 рёбер $E \rightarrow R$ и 49 дополнительных рёбер с `loop-vectorize` *содержательный каузальный граф* содержит **372** ребра.

6.3.2 Топ-10 содержательных рёбер

В таблице 2 представлены 10 рёбер содержательного графа с наибольшим $|\widehat{ATE}|$.

Таблица 2: Топ-10 рёбер содержательного каузального графа по $|\widehat{ATE}|$.

#	Ребро	$\widehat{ATE}$
1	<code>gvn</code> $\rightarrow$ <code>instcombine</code>	+0,034
2	<code>gvn</code> $\rightarrow$ <code>dse</code>	+0,026
3	<code>early-cse-memssa</code> $\rightarrow$ <code>instcombine</code>	+0,024
4	<code>newgvn</code> $\rightarrow$ <code>instcombine</code>	+0,024
5	<code>gvn</code> $\rightarrow$ <code>attributor</code>	+0,024
6	<code>early-cse-memssa</code> $\rightarrow$ <code>dse</code>	+0,023
7	<code>newgvn</code> $\rightarrow$ <code>dse</code>	+0,022
8	<code>licm</code> $\rightarrow$ <code>dse</code>	+0,022
9	<code>licm</code> $\rightarrow$ <code>instcombine</code>	+0,018
10	<code>loop-reduce</code> $\rightarrow$ <code>instcombine</code>	+0,017

Все десять рёбер топ-10 представляют собой канонические инженерные паттерны, применяемые в стандартных пайплайнах LLVM: проходы устранения дубликатов и нумерации значений (`gvn`, `newgvn`, `early-cse-memssa`) предшествуют комбинированию инструкций (`instcombine`) и удалению мёртвых записей (`dse`); проход выноса инвариантов цикла (`licm`) также предшествует `dse` и `instcombine`; снижение силы индукционных переменных (`loop-reduce`) предшествует `instcombine`. Эти паттерны эмпирически воспроизводят давно сложившиеся в инженерной практике рекомендации по

упорядочиванию оптимизаций.

Структура содержательного графа визуализирована на рисунке 2: показаны 50 наиболее сильных рёбер. В отличие от полного графа (рисунк 1), в содержательном графе устранена доминирующая роль раздваивающих проходов; источники с наибольшим числом исходящих рёбер — проходы устранения дубликатов и нумерации значений.

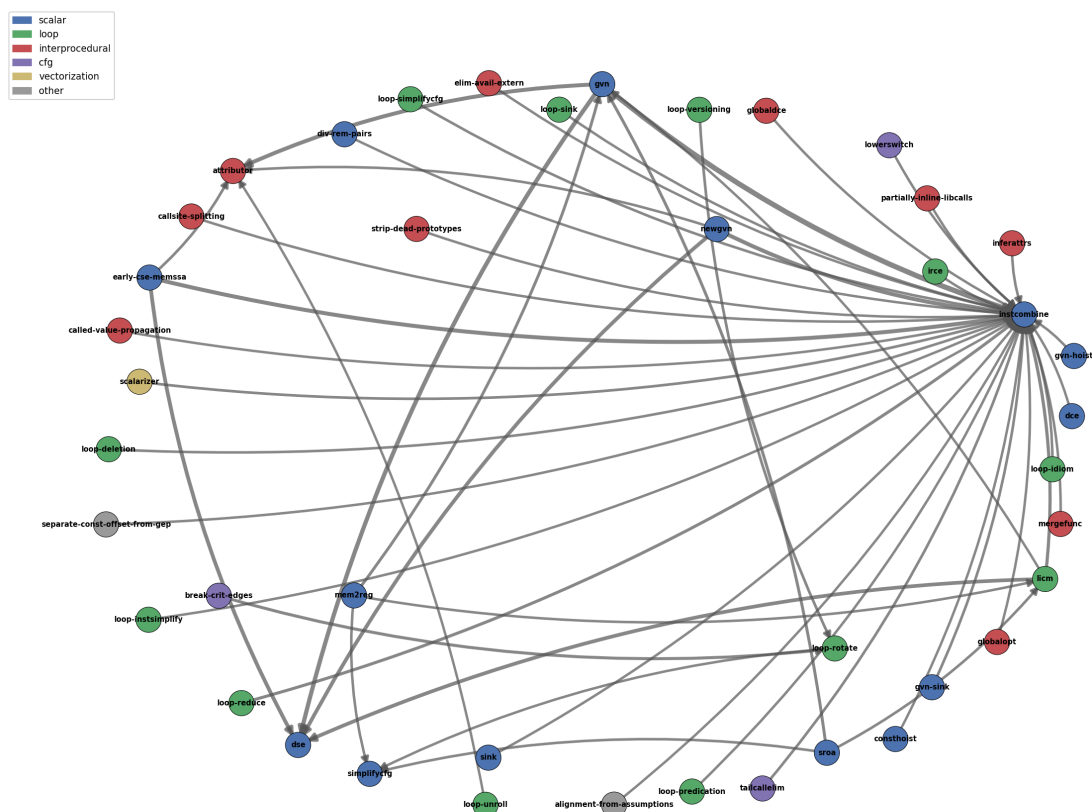


Рис. 2: Топ-50 рёбер глобального содержательного каузального графа
(372 ребра).

6.3.3 Согласованность с пайплайном -0z

Топ-рёбра содержательного графа сопоставлены с фактическим порядком проходов в пайплайне -Oz LLVM 16. В -Oz наблюдается следующий харак-

терный фрагмент:

$$\dots \rightarrow \text{gvn} \rightarrow \text{sccp} \rightarrow \text{bdce} \rightarrow \text{instcombine} \rightarrow \\ \rightarrow \text{jump-threading} \rightarrow \text{adce} \rightarrow \text{memcpropt} \rightarrow \text{dse} \rightarrow \dots,$$

что в точности соответствует обнаруженным каузальным рёбрам $\text{gvn} \rightarrow \text{instcombine}$ и $\text{gvn} \rightarrow \text{dse}$. Это подтверждает результаты каузального вывода: статистический анализ случайных перестановок из 5,17 миллиона прогонов воспроизводит инженерные решения, заложенные разработчиками LLVM в стандартный пайплайн оптимизации.

В дальнейшем изложении настоящей главы (per-class и per-cluster графы, гетерогенность CATE) используется содержательный граф из 372 рёбер.

6.4 Каузальные графы на уровне доменных классов

Применение метода и фильтрации $E \rightarrow R$ (см. раздел 6.3) к каждому из 8 доменных классов программ даёт 8 содержательных каузальных графов. Размер каждого графа представлен в таблице 3.

Число содержательных рёбер в per-class графах превышает число рёбер глобального содержательного графа (372) для большинства классов. Это объясняется тем, что эффекты, усреднённые до незначимых величин на глобальном уровне, могут оставаться значимыми в пределах однородной по характеру вычислений подгруппы. Классы `loop_bench` (тесты циклов) и `numeric` (численные алгоритмы) дают наибольшее число содержательных рёбер: эффекты порядка проходов наиболее выражены для программ, где доминируют циклы и арифметические операции.

Таблица 3: Размер содержательных per-class каузальных графов,
вариант 1.

Класс	Программ в классе	Содержательных рёбер
crypto	49	468
general	49	592
loop_bench	36	710
multimedia	61	391
numeric	82	687
pointer_data	33	433
string_parse	76	379
systems	131	373

6.5 Каузальные графы на уровне структурных кластеров

Применение метода и фильтрации $E \rightarrow R$ к 6 структурным кластерам даёт 6 содержательных per-cluster графов. Размеры представлены в таблице 4.

Таблица 4: Размер содержательных per-cluster каузальных графов,
вариант 1.

Кластер	Программ в кластере	Содержательных рёбер
cluster_0	169	574
cluster_1	137	345
cluster_2	22	309
cluster_3	28	322
cluster_4	137	334
cluster_5	24	53

Малые однородные кластеры (`cluster_2`, `cluster_3`, `cluster_5`) дают компактные графы — меньшее число программ ограничивает число

пар, по которым достигается статистическая значимость. Крупные кластеры (`cluster_0`, `cluster_4`) дают существенно больше содержательных рёбер за счёт большего размера выборок.

Сравнение с per-class классификацией показывает, что группировки по доменному характеру и по структуре IR не совпадают: индекс Adjusted Rand между двумя разбиениями составляет 0,097, индекс нормированной взаимной информации (NMI) — 0,193. Тем не менее, наблюдаются согласованности: class `loop_bench` (36 программ TSVC) попадает в `cluster_1` (control-flow heavy) на 100 %, а около 29 % программ class `crypto` попадают в `cluster_2` (logic/crypto). Таким образом, автоматическая кластеризация частично воспроизводит ручную классификацию по характеру вычислений.

6.6 Гетерогенность каузальных эффектов (CATE)

Для каждого из 372 рёбер глобального содержательного каузального графа вычислены условные средние эффекты воздействия (CATE) по 8 доменным классам и 6 структурным кластерам. Анализ CATE позволяет выявить, какие рёбра являются *универсальными* (эффект стабилен на всех подгруппах), а какие — *специализированными* (эффект существенно различается между подгруппами).

6.6.1 Рёбра с противоположным знаком на подгруппах

В таблице 5 представлено число рёбер содержательного каузального графа, у которых знак CATE на отдельной подгруппе противоположен знаку глобального $\widehat{ATE}$.

На уровне доменных классов наибольшее число расхождений со знаком глобального графа наблюдается в `loop_bench` — 7 % рёбер. На уровне структурных кластеров наибольшее число расхождений — 29 % рёбер в

Таблица 5: Число рёбер содержательного графа со сменой знака CATE на подгруппах.

Подгруппа	Рёбер со сменой знака	Доля от 372
loop_bench (per-class)	26	7 %
cluster_5 (per-cluster)	109	29 %

`cluster_5` (программы с доминированием вызовов функций).

Полученные оценки подтверждают, что глобальный каузальный граф не описывает оптимальный порядок проходов универсально: для существенной части пар проходов оптимальный порядок зависит от характера программы. Это обосновывает построение специализированных каузальных графов на уровне домена и структуры IR.

Гетерогенность каузальных эффектов наглядно представлена на рисунке 3: для топ-30 рёбер содержательного графа (строки) и подгрупп (столбцы) приведены значения CATE с цветовой кодировкой. Отрицательные значения (рёбра со сменой знака) выделяются цветом, противоположным основному.

Распределение числа рёбер со сменой знака CATE по всем подгруппам (8 доменных классов + 6 структурных кластеров) представлено на рисунке 4. Видно, что доминирующая гетерогенность сосредоточена в одном кластере (`cluster_5`), тогда как в большинстве классов и кластеров доля рёбер со сменой знака не превышает нескольких процентов.

6.7 Сравнение двух формулировок воздействия

Применение метода к формулировке воздействия 2 (соседство) на глобальном уровне даёт **44 ребра** — более чем на порядок меньше, чем 608 рёбер варианта 1 (относительный порядок). Все 44 ребра варианта 2 имеют $|\widehat{\text{ATE}}| > 0,005$ ещё до применения порога: для соседства редко возникают

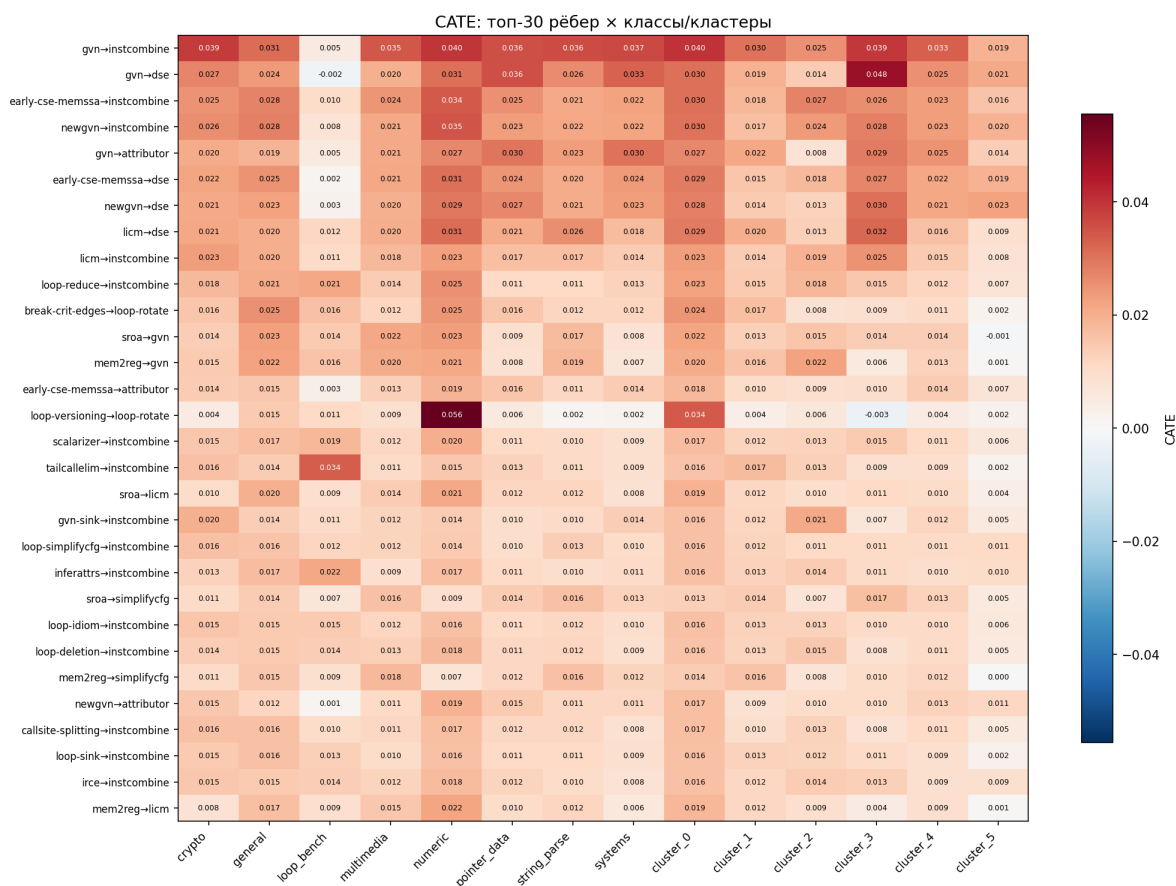


Рис. 3: Тепловая карта CATE для топ-30 рёбер содержательного каузального графа.

пары со статистически значимым, но крайне малым эффектом.

Содержательное различие двух формулировок:

- **Вариант 1 (относительный порядок)** ловит *глобальные* зависимости: ребро в графе указывает, что наличие прохода A где-то раньше прохода B систематически улучшает метрику относительно противоположного порядка, независимо от расположения остальных проходов между ними.
- **Вариант 2 (соседство)** ловит *локальные синергии*: ребро указывает, что непосредственное соседство $[A, B]$ систематически даёт лучший результат, чем все остальные конфигурации с теми же двумя проходами.

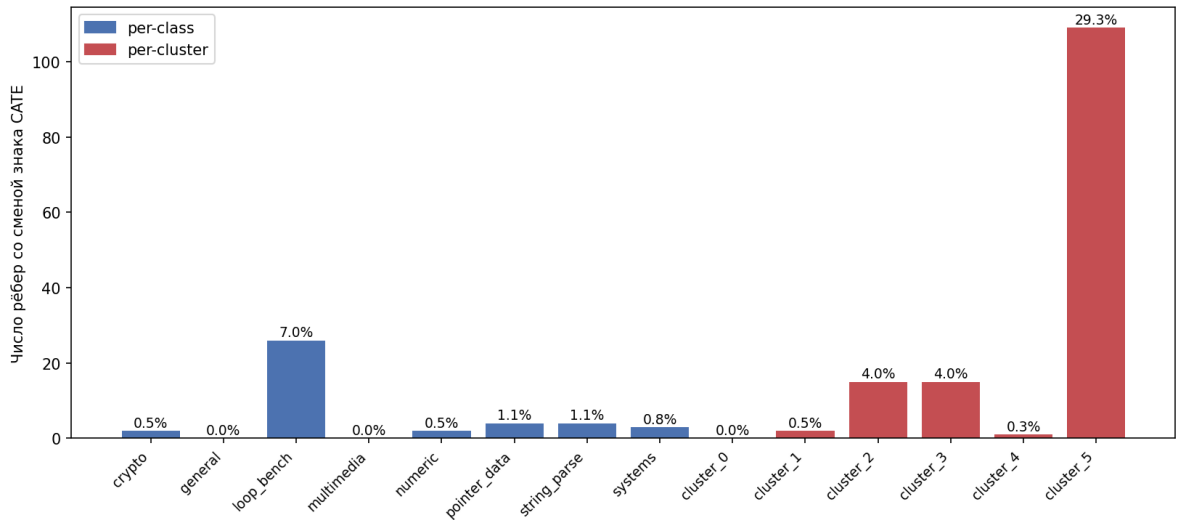


Рис. 4: Число рёбер содержательного графа со сменой знака CATE по подгруппам.

Различие в числе рёбер (608 против 44) интерпретируется следующим образом: подавляющее большинство каузальных зависимостей между проходами LLVM имеют *глобальный*, а не локальный характер — эффект порядка пары проходов проявляется даже при наличии других проходов между ними. Локальные синергии (требующие непосредственного соседства) составляют меньшинство выявленных эффектов.

6.8 Сравнение каузального и корреляционного графов

Корреляционный граф, построенный по методике раздела 4.6, на глобальном уровне содержит **5124 ребра** — более чем в 8 раз больше, чем каузальный граф варианта 1 (608 рёбер) и более чем в 100 раз больше, чем каузальный граф варианта 2 (44 ребра, та же формулировка воздействия). Из 5124 корреляционных рёбер лишь 376 присутствуют также в каузальном графе варианта 1. Доля корреляционных рёбер, не имеющих каузального соответствия:

$$\frac{5124 - 376}{5124} \approx 92,7 \, \%.$$

Полученная оценка интерпретируется как нижняя граница доли лож-

ных корреляций в стандартном корреляционном анализе зависимостей между проходами для рассматриваемой метрики качества кода. Визуальное сравнение каузального и корреляционного графов представлено на рисунке 5.

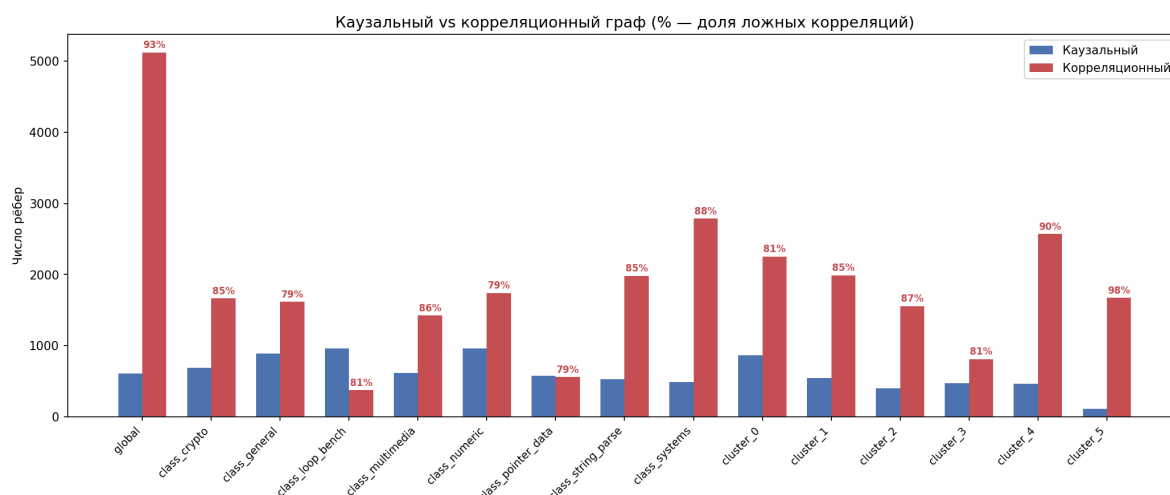


Рис. 5: Сравнение каузального (608 рёбер) и корреляционного (5124 ребра) графов на одних и тех же данных.

6.8.1 Сравнение по подгруппам

Доля ложных корреляционных рёбер варьируется по подгруппам в диапазоне от 78,7 % (класс **general**) до 98,1 % (**cluster_5**). Различие отражает разную внутреннюю однородность подгрупп: чем более разнородна подгруппа, тем больше совпадений случайного характера между корреляционным и каузальным графами; чем более однородна — тем выше доля корреляций без каузальной природы.

6.8.2 Содержательная интерпретация расхождения

Расхождение каузального и корреляционного графов содержательно объясняется устройством компиляторных пайплайнов. Если проход A полезен в составе типичной последовательности оптимизаций, то выигрышные последовательности часто содержат A в одной из позиций. Любой проход B ,

оказавшийся рядом с A в этих последовательностях, окажется статистически связан с выигрышностью, даже если его собственный вклад в эффект пренебрежим. Корреляционный анализ не способен отделить такие косвенные связи от прямых каузальных. Рандомизация порядка проходов в дизайне эксперимента и формальный аппарат каузального вывода устраняют эту неоднозначность и позволяют выделить пары проходов с реальным каузальным влиянием.

7 Заключение

В настоящей работе предложен и реализован метод обнаружения каузальных зависимостей между парами оптимизирующих проходов компилятора LLVM на основе формального аппарата модели потенциальных исходов и рандомизированного эксперимента. Метод применён к набору из 89 оптимизирующих проходов LLVM 16 и 517 тестовых программ; собран эмпирический корпус из 5,17 миллиона прогонов случайных последовательностей; построены и проанализированы каузальные графы зависимостей на трёх уровнях агрегации.

Основные результаты

1. **Методологический результат.** Показано, что случайные перестановки оптимизирующих проходов LLVM образуют рандомизированный эксперимент в смысле Неймана–Рубина: для каждой пары проходов условие независимости назначения воздействия от потенциальных исходов выполнено в силу симметрии равномерной выборки без повторений. Следовательно, наивная оценка среднего эффекта воздействия совпадает с истинным каузальным эффектом без необходимости применения алгоритмов поиска структуры (РС-алгоритм, FCI и аналогичные).
2. **Каузальный граф зависимостей.** Построен глобальный каузальный граф из 608 рёбер для формулировки относительного порядка и 44 рёбер для формулировки соседства, а также 14 специализированных графов на уровне доменных классов и структурных кластеров. Структура графа воспроизводит инженерные паттерны стандартного пайплайна -Oz: фрагмент `gvn` $\rightarrow$ `instcombine` $\rightarrow$ `dse`, обнаруженный методом, в точности соответствует фактическому порядку проходов в

-0z.

3. **Количественная оценка ложных корреляций.** На одних и тех же данных построен корреляционный граф из 5124 рёбер. Из них лишь 376 рёбер присутствуют также в каузальном графе; доля корреляционных рёбер без каузального соответствия составляет 92,7 %. Полученная оценка служит нижней границей доли ложных корреляций при использовании стандартного корреляционного подхода для анализа зависимостей между проходами.
4. **Гетерогенность каузальных эффектов.** Анализ условных средних эффектов воздействия (CATE) для рёбер содержательного графа выявил существенную гетерогенность эффектов по типам программ: до 29 % рёбер глобального графа меняют знак на отдельных структурных кластерах. Это обосновывает невозможность универсального оптимального порядка проходов и необходимость специализированных пайплайнов для разных типов программ.
5. **Сравнение формулировок воздействия.** Две формулировки — относительный порядок и непосредственное соседство — выявляют различные классы зависимостей: глобальные (608 рёбер) и локальные синергии (44 ребра) соответственно. Подавляющее большинство выявленных каузальных зависимостей имеют глобальный, а не локальный характер.
6. **Собранные датасеты.** 517 бенчмарков, скомпилированных в бинарное представление LLVM IR, и 5,17 миллиона прогонов с измеренной метрикой качества кода представляют самостоятельный научный вклад работы и могут быть использованы в последующих исследованиях.

Соответствие задачам работы

Все шесть задач, поставленных в **Главе 1**, выполнены: сформирован набор оптимизирующих проходов и тестовых программ (Глава 5, Приложение А); спроектирован и проведён масштабный рандомизированный эксперимент (Глава 5); разработан программный pipeline каузального анализа (Глава 4); построены каузальные графы на трёх уровнях агрегации (Глава 6, разделы 6.2–6.5); исследована гетерогенность эффектов с помощью CATE и сравнены две формулировки воздействия (разделы 6.6, 6.7); построен корреляционный граф и количественно сравнён с каузальным (раздел 6.8). Все пять положений, выносимых на защиту, обоснованы соответствующими результатами Главы 6.

Ограничения метода

1. **Асимметрия метрики.** Выбранная метрика (число IR-инструкций) обладает структурным смещением относительно положения прохода в последовательности: код-раздувающие проходы систематически дают больший эффект при раннем расположении, код-сокращающие — при позднем расположении в роли цели. Это смещение не полностью отделимо от каузального эффекта на основании только величины $\widehat{ATE}$ и потребовало введения дополнительной фильтрации $E \rightarrow R$ для построения содержательного графа.
2. **Отключающие эффекты.** Часть рёбер каузального графа отражает эффекты «отключения»: проход в позиции, где его предусловия не выполнены, не выполняет работы, и каузальный эффект регистрируется как уменьшение размера кода за счёт несрабатывания, а не за счёт удачного упорядочивания. Различение продуктивных и отключающих рёбер потребовало экспериментальной проверки на синтети-

ческих примерах и эвристического исключения наиболее проблемного прохода (`loop-vectorize`).

3. **Малая величина отдельных эффектов.** Медианная величина $|d|$ Кона для значимых рёбер составляет около 0,017, что на порядок меньше порога «малого эффекта». Содержательная ценность результатов состоит не в крупных эффектах отдельных пар, а в структуре их совокупности.

Направления дальнейших исследований

1. **Прямое детектирование срабатывания проходов.** Введение instrumentation, регистрирующей факт срабатывания каждого прохода (число фактически выполненных трансформаций), позволит формально разделить продуктивные и отключающие эффекты и снять необходимость в эвристической фильтрации $E \rightarrow R$.
2. **Применение других метрик качества кода.** Применение метода к метрике времени выполнения вместо размера кода устранил асимметрию, связанную с размером IR , и потенциально выявит другие классы каузальных зависимостей — в том числе паттерны $R \rightarrow E$ (подготовка перед раздуванием), которые в настоящей работе не обнаружены из-за структурного смещения метрики.
3. **Построение оптимизированных последовательностей.** Полученные каузальные графы могут служить основой для алгоритмов построения специализированных последовательностей проходов под конкретные программы — например, через жадный поиск по графу или направленный поиск с использованием рёбер графа в качестве ограничений.

Список литературы

- [1] LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation / Chris Lattner, Vikram Adve // Proceedings of the International Symposium on Code Generation and Optimization (CGO). — 2004. — Pp. 75–88.
- [2] A Survey on Compiler Autotuning using Machine Learning / Amir H. Ashouri, William Killian, John Cavazos et al. // *ACM Computing Surveys*. — 2018. — Vol. 51, no. 5. — P. Article 96.
- [3] Optimizing for reduced code space using genetic algorithms / Keith D. Cooper, Philip J. Schielke, Devika Subramanian // Proceedings of the ACM SIGPLAN Workshop on Languages, Compilers, and Tools for Embedded Systems (LCTES). — ACM, 1999. — Pp. 1–9.
- [4] Finding effective compilation sequences / L. Almagor, Keith D. Cooper, Alexander Grosul et al. // Proceedings of the ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES). — ACM, 2004. — Pp. 231–239.
- [5] Exhaustive optimization phase order space exploration / Prasad A. Kulkarni, David B. Whalley, Gary S. Tyson, Jack W. Davidson // Proceedings of the International Symposium on Code Generation and Optimization (CGO). — IEEE, 2006. — Pp. 306–318.
- [6] Mitigating the Compiler Optimization Phase-Ordering Problem using Machine Learning / Sameer Kulkarni, John Cavazos // Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA). — ACM, 2012. — Pp. 147–162.

- [7] Learning Compiler Pass Orders using Coreset and Normalized Value Prediction / Youwei Liang, Kevin Stone, Ali Shameli et al. // Proceedings of the 40th International Conference on Machine Learning (ICML). — Vol. 202 of *PMLR*. — 2023.
- [8] ECCO: Evidence-Driven Causal Reasoning for Compiler Optimization / Haolin Pan, Lianghong Huang, Jinyuan Dong et al. // *arXiv preprint arXiv:2602.00087*. — 2026.
- [9] Milepost GCC: Machine Learning Enabled Self-tuning Compiler / Grigori Fursin, Yuriy Kashnikov, Abdul Wahid Memon et al. // *International Journal of Parallel Programming*. — 2011. — Vol. 39, no. 3. — Pp. 296–327.
- [10] *Haj-Ali, Ameer*. Machine Learning in Compiler Optimization: Ph.D. thesis / University of California, Berkeley. — 2020.
- [11] MLGO: a Machine Learning Guided Compiler Optimizations Framework / Mircea Trofin, Yundi Qian, Eugene Brevdo et al. // *arXiv preprint arXiv:2101.04808*. — 2021.
- [12] *Pearl, Judea*. The Book of Why: The New Science of Cause and Effect / Judea Pearl, Dana Mackenzie. — Basic Books, 2018.
- [13] Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies / Donald B. Rubin // *Journal of Educational Psychology*. — 1974. — Vol. 66, no. 5. — Pp. 688–701.
- [14] *Pearl, Judea*. Causality: Models, Reasoning, and Inference / Judea Pearl. — 2 edition. — Cambridge University Press, 2009.
- [15] Causal inference in statistics: An overview / Judea Pearl // *Statistics Surveys*. — 2009. — Vol. 3. — Pp. 96–146.

- [16] *Imbens, Guido W.* Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction / Guido W. Imbens, Donald B. Rubin. — Cambridge University Press, 2015.
- [17] Unicorn: Reasoning about Configurable System Performance through the Lens of Causality / Md Shahriar Iqbal, Rahul Krishna, Mohammad Ali Javidian et al. // Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys). — 2022. — Pp. 199–217.
- [18] Causality in Configurable Software Systems / Clemens Dubsclaff, Kallistos Weis, Christel Baier, Sven Apel // Proceedings of the 44th International Conference on Software Engineering (ICSE). — 2022.
- [19] Causal testing: understanding defects' root causes / Brittany Johnson, Yuriy Brun, Alexandra Meliou // Proceedings of the 42nd International Conference on Software Engineering (ICSE). — 2020.
- [20] Adaptive Optimizing Compilers for the 21st Century / Keith D. Cooper, Devika Subramanian, Linda Torczon // *The Journal of Supercomputing*. — 2002. — Vol. 23, no. 1. — Pp. 7–22.
- [21] Machine Learning in Compiler Optimization / Zheng Wang, Michael O'Boyle // *Proceedings of the IEEE*. — 2018. — Vol. 106, no. 11. — Pp. 1879–1901.
- [22] OpenTuner: An Extensible Framework for Program Autotuning / Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni et al. // Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT). — 2014. — Pp. 303–316.
- [23] AutoPhase: Juggling HLS Phase Orderings in Random Forests with Deep Reinforcement Learning / Ameer Haj-Ali, Qijing Jenny Huang, John Xiang et al. // Proceedings of Machine Learning and Systems (MLSys). — 2020.

- [24] CompilerGym: Robust, Performant Compiler Optimization Environments for AI Research / Chris Cummins, Bram Wasti, Jiadong Guo et al. // Proceedings of the International Symposium on Code Generation and Optimization (CGO). — 2022.
- [25] ProGraML: A Graph-based Program Representation for Data Flow Analysis and Compiler Optimizations / Chris Cummins, Zacharias V. Fisches, Tal Ben-Nun et al. // Proceedings of the 38th International Conference on Machine Learning (ICML). — 2021.
- [26] On the Application of Probability Theory to Agricultural Experiments. Essay on Principles. Section 9 / Jerzy Splawa-Neyman, Dorota M. Dabrowska, Terence P. Speed // *Statistical Science*. — 1990. — Vol. 5, no. 4. — Pp. 465–472. — Перевод оригинальной работы 1923 года.
- [27] *Hernán, Miguel A.* Causal Inference: What If / Miguel A. Hernán, James M. Robins. — Chapman & Hall/CRC, 2020.
- [28] An Introduction to Causal Inference / Judea Pearl // *The International Journal of Biostatistics*. — 2010. — Vol. 6, no. 2.
- [29] Methods and tools for causal discovery and causal inference / Ana Rita Nogueira, Andrea Pugnana, Salvatore Ruggieri et al. // *WIREs Data Mining and Knowledge Discovery*. — 2022. — Vol. 12, no. 2. — P. e1449.
- [30] *Spirtes, Peter.* Causation, Prediction, and Search / Peter Spirtes, Clark Glymour, Richard Scheines. — 2 edition. — MIT Press, 2000.
- [31] *LLVM Project.* LLVM Language Reference Manual. — <https://llvm.org/docs/LangRef.html>. — 2024.

- [32] *LLVM Project*. LLVM’s Analysis and Transform Passes. — <https://llvm.org/docs/Passes.html>. — 2024.
- [33] The generalization of “Student’s” problem when several different population variances are involved / Bernard L. Welch // *Biometrika*. — 1947. — Vol. 34, no. 1/2. — Pp. 28–35.
- [34] The Probable Error of a Mean / William S. Student (Gosset // *Biometrika*. — 1908. — Vol. 6, no. 1. — Pp. 1–25.
- [35] Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing / Yoav Benjamini, Yosef Hochberg // *Journal of the Royal Statistical Society, Series B*. — 1995. — Vol. 57, no. 1. — Pp. 289–300.
- [36] *Cohen, Jacob*. Statistical Power Analysis for the Behavioral Sciences / Jacob Cohen. — 2 edition. — Lawrence Erlbaum Associates, 1988.
- [37] *LaC-DCC*. LaC-DCC Benchmarks Collection. — <https://github.com/lac-dcc/>. — 2024.
- [38] ANGHABENCH: A Suite with One Million Compilable C Benchmarks for Code-Size Reduction / Anderson Faustino da Silva, Bruno Conde Kind, José Wesley de Souza Magalhães et al. // Proceedings of the International Symposium on Code Generation and Optimization (CGO). — 2021. — Pp. 378–390.

Приложение А. Полный список оптимизирующих проходов

В работе используется набор из 89 оптимизирующих проходов LLVM 16, перемешиваемых в случайные последовательности. Все проходы проверены на работоспособность в legacy pass manager как по отдельности, так и в составе полного набора. Источники формирования набора:

- проходы пайплайна -Oz в LLVM 16 (33 прохода);
- дополнительные проходы из стандартного арсенала LLVM, охватывающие семантически значимые категории (14 проходов);
- базовые проходы канонизации SSA и циклов (4 прохода);
- проходы инлайнинга, векторизации и скалярных оптимизаций (23 прохода);
- специализированные проходы преобразования циклов и вспомогательные проходы (15 проходов).

Проходы пайплайна -Oz (33)

bdce, ipsccp, correlated-propagation, mem2reg, sroa, instsimplify, constmerge, function-attrs, deadargelim, globaldce, elim-avail-extern, mergefunc, instcombine, simplifycfg, tailcallelim, reassociate, memcpyopt, sccp, dce, adce, dse, jump-threading, loop-idiom, loop-deletion, gvn, gvn-hoist, gvn-sink, newgvn, early-cse, mergereturn, globalopt, strip-dead-prototypes, indvars.

Дополнительные проходы из стандартного арсенала LLVM

(14)

Эти проходы добавлены к набору для покрытия семантически значимых категорий, не полностью представленных проходами пайплайна -Oz.

Анализ атрибутов функций (4):

attributor, inferattrs, rpo-function-attrs, called-value-propagation.

Цикловые преобразования (3):

loop-reduce, loop-unroll, loop-rotate.

Скалярные оптимизации (4):

mergeicmps, separate-const-offset-from-gep, libcalls-shrinkwrap, flattencfg.

Аннотации и подсказки (3):

partial-inliner, lower-expect, alignment-from-assumptions.

Базовые проходы канонизации (4)

licm, loop-simplify, div-rem-pairs, float2int.

Инлайнинг, векторизация, скалярные оптимизации (23)

inline, sink, loop-vectorize, slp-vectorizer, callsite-splitting, dfa-jump-threading, consthoist, mldst-motion, slsr, loop-interchange, loop-fusion, loop-distribute, loop-load-elim,

loop-sink, loop-unroll-and-jam, hotcoldsplit, irce,
nary-reassociate, scalarizer, vector-combine, always-inline,
globalsplit, lowerswitch.

Преобразования циклов и вспомогательные (15)

early-cse-memssa, simple-loop-unswitch, loop-versioning,
loop-flatten, loop-reroll, loop-instsimplify, loop-simplifycfg,
loop-predication, speculative-execution,
partially-inline-libcalls, load-store-vectorizer, iroutliner,
global-merge, break-crit-edges, unreachableblockelim.